

Supporting Decisions in a Near-Unpredictable Game

Version 5.205.0 · Last updated 2026-08-28

5.205.0 — THE MEDICHAM SPRINT IS PAUSED AND THE DOCUMENTS ARE UNFROZEN. THE GATE IS NOT OPEN, AND FIVE OF ITS EIGHT CLAUSES ARE WITHHELD RATHER THAN REPORTED.

On 2026-08-10 the living-docs rule was deferred by the owner for the duration of the MEDICHAM correctness sprint: each fix wrote one row to a running log and the batch was to be written up when the gate closed. Will paused the sprint on 2026-08-28 and this pass discharges the debt. The log held 274 rows; `CHANGELOG.md` carries 233 releases between 3.99.1 and 5.204.0, every one of them a sprint row; `docs/_reports/` carries 189 dated accounts. **The per-release detail lives there and is not restated here.** The log file is deleted and the full living-docs rule is re-armed, which is what deleting it means.

THE THROUGH-LINE IS NOT A LIST OF FIXES. It is that the INSTRUMENT was the defect at least six times, and that the largest single event of the sprint RAISED the visible defect count rather than lowering it. Named, each with a report: the deliberate roster's red demonstrations had never been written into its artifact, so a gate clause that reads them had nothing to fail on; of thirty accusations then classified, twenty-three were a miswritten demonstration, seven were a wrong rule and exactly one was the engine failing to react; eighteen expired "shown red" certificates were being published as a broken simulator; three board-material games attributed to a spread secondary were the ruler; a golden-master check was dead from its second load and had been read as a spread-immunity damage defect; a grep-based identity check was red on a file that does the thing it checked for. The project's own rule — *suspect the instrument before the engine* — was earned, not assumed.

THE EVENT DIE WAS TRANSLATING RATHER THAN RE-DRAWING, AND THAT IS WHY TWO ENGINES HAD BEEN AGREEING BY ACCIDENT. `midEventHash` and `midHash` ended on `h = Math.imul(h ^ c, 0x01000193)` with no finalising mix, and the last field of every draw address is the arrival index `nth`. Changing only the final character therefore TRANSLATES the hash instead of re-drawing it:

$v(nth=d) - v(nth=0) = (((A \oplus c_d) - (A \oplus c_0)) \cdot P \bmod 2^{32}) / 2^{32}$, with `P = 0x01000193` the FNV-1a prime from the source line above — a constant of the hash, not a measurement.

A one-digit index differs in its low four bits only, which bounds the shift. Two-digit indices mix correctly, which is why nothing caught it for weeks.

quantity	bare FNV-1a	with a finalising mix	independent
max circular shift, <code>v(nth=d)</code> against <code>v(nth=0)</code>	0.0351571	0.4999829	~0.5
consecutive arrivals sharing a 16-bucket damage index	89.5%	6.2%	6.25%
distinct damage indices from a ten-hit address	1.75	7.60	7.56
two same-turn residual half-coins landing the same way	99.1%	48.5%	50%
lag-1 autocorrelation down one address axis	0.8873	-0.0024	~0
marginal hit rate on that sweep	0.9214	0.8992	0.9

The last row is the lesson, and it is a general one about rulers. The marginal was always fine, and the assertion watching this axis measured only the marginal. A die can be uniform in aggregate and almost perfectly predictable one step at a time. Fixing it moved the whole-game differential from 3 to 14

games and the board-material subset from 1 to 12 — an instrument repaired, not a regression introduced. The prediction was written before the run and published as written: whole-game 3 to between 3 and 15 (14, inside), board-material 1 to between 1 and 8 (12, **outside and above**).

EVERY FIGURE IN THIS PROJECT THAT WAS MEASURED BEFORE 2026-08-27 AND PASSED THROUGH THAT DIE IS VOID, NOT STALE. Void means the two engines were compared over a narrower slice of outcome space than the comparison claimed to cover, so an agreement is not evidence of agreement. Every such figure that appears in the sections below is retained as DATED HISTORY, in this project's standing practice of never silently rewriting a prior conclusion — and **none of them may be cited as current**. The current state of any of them is what `node engine/status.js` prints today, and where that prints nothing, nothing is known.

WHAT THE GATE SAYS TODAY, AND WHY MOST OF IT IS BLANK. `engine/quarantine.js` computes the MEDICHAM gate from artifacts. Read from `data/quarantine-stamp.json`: `gate_open` is **false**, with five failing clauses — the three deliberate-roster stages, the whole-game differential, and the staged-mechanics comparison. Those five are **WITHHELD, not annotated**: each artifact was measured against engine release `5f3f7141227c` and the tree now hashes to a different release, so every count in them describes a simulator that is not the one on disk. A figure printed beside a warning is the failure this project has already paid for, so no rate, no diverged count and no roster column is reproduced here.

The cause of that stranding is measured and is not an engine change. Exactly one of the twenty-six frozen sources moved: `data/tags.json`, whose stored copy inside the release is byte-identical to the working tree after newline normalisation and deep-equal when both are parsed as JSON. A checkout under `core.autocrlf = true` rewrote a generated LF file as CRLF between the measurement and now. `docs/ENGINE.md` records the identical event on 2026-08-26. **This does not make the withheld numbers quotable.** The remedy is a re-run over the bytes a checkout actually produces, and until that run exists the clauses say nothing.

WHAT IS NOT WITHHELD, AND EXACTLY HOW FAR IT REACHES.

- **The behavioural census.** Read from `data/mechanics-census.json`: **780 probed, 780 live, 0 missing**, 780 armed and 0 unarmed, 0 threw and 0 hollow. This is a lab: one deliberately staged scenario per mechanic, regardless of whether anybody plays it. It answers *is this correct*. It does not answer *does this matter*, which is the pinned pool's question.
- **The damage differential.** Read from `data/engine-diff.json`: 6000 compared, 6000 agreed, 0 disagreed, and 0 disagreements at the midpoint and at each of the sixteen indices of the damage band separately, never pooled. **THIS FIGURE IS NARROWER THAN IT LOOKS AND MUST NOT BE QUOTED AS A GENERAL AGREEMENT.** The artifact's own `scope` field limits it to *"damage only, no items or abilities"*; turn order, status duration and switching are not attempted. It also records `skipped_multihit` at 134 and `skipped_ability_multihit` at 17: the harness calls the authority's single-hit entry point rather than the volley loop, so **it has never applied a multi-hit move**, and the four multi-hit defects fixed during this sprint were invisible to it by construction.

THE BOARD COMPARISON READS 33 OF 80 LEAVES, AND THAT IS THE SINGLE MOST IMPORTANT CAVEAT IN THIS DOCUMENT. "The boards match" is a claim about the leaves the comparison actually reads. `tests/probe_uncompared_leaves.js` derives, over 500 legal moves, 201 abilities with a legal carrier and 148 legal items, every leaf a legal mechanic can write: 80 distinct leaves, of which **33 are compared, 4 are explicitly declared uncompared, and 43 are in NEITHER list**. Twenty-five of those 43 can be standing on the board at the turn boundary where the comparison is taken. `board_state.js` states the consequence in its own words — *an unlisted omission reads exactly like agreement* — and twice in one night a mechanic's verdict proved unearned for exactly this reason. Any sentence in this paper of the form "the boards agree" is bounded by that 33.

QUARANTINE HAS NOT LIFTED, AND THE TWO FACTS UNDER THAT SENTENCE ARE BOTH TRUE. The computed condition — the differential clean and the roster clean across items, abilities and moves — is not met, so every artifact downstream of the simulator stays withheld. Will's own bar, ruled on 2026-08-22, is narrower: board-material zero with narration as a separate gate afterwards, plus a clean roster. The last measurements taken before the stranding met that narrower bar. **These two facts are not resolved here, and this paper does not resolve them:** the ruling that would re-cut the gate to test board-material rather than whole-game divergence was never implemented, so the gate still computes the wider condition and still reads shut.

EVERY FIGURE DOWNSTREAM OF THE SIMULATOR IS RE-RUNNABLE, NOT TRUE. A quarantined number does not become correct when MEDICHAM becomes correct. It becomes eligible to be measured again. That applies without exception to leaf calibration, the rollout rungs R1 to R4, the exploitability results, the MAG and joint weight vectors, and every head-to-head in the sections below. The re-run list is ROADMAP #57 and the refit is owed, gated on the engine rather than on compute.

3.98.0 — FIVE OF SIX SOURCES OF PRIORITY REFUSAL WERE CORRECT; THE SIXTH TOLD ITSELF APART BY NAME. A +1 priority attack was staged against each source in turn on the frozen release. Armor Tail, Dazzling, Queenly Majesty and Psychic Terrain all refused it (0 damage against a control of 25); Wide Guard let it through, correctly, because it stops spread moves rather than priority ones; **Quick Guard let it through too**, and that is a defect on 927 corpus clicks. The cause is that `quickguard` and `wideguard` carry *byte-identical tag lists* — `priority`, `neverMisses`, `oneTurnGuard`, `statusCategory` — so three sites in the simulator separated them by spelling: the action classifier (`if(id==='wideguard')`), which sent Quick Guard to the no-op `{kind:'pass'}` branch), the move-legality filter in `buildMon` (which deleted Quick Guard from a declared body before any turn ran), and the field state itself (a boolean pair whose *name* was the only record of what it guarded against). The parameter that separates them — `oneTurnGuard.blocks` , derived by `tag_dex` from each move's own `condition.onTryHit` — has been in `data/tags.json` since the tag was written and nothing read it, so `engine/tag_dex.js` **did not change and no artifact was regenerated.** The refusal is wired onto the same gate the ability sources already use, above the action-kind dispatch, which is what makes a Prankster-boosted status move refusable (Showdown tests the *final* priority: `if (move.priority <= 0.1) return`). Feint and the other thirteen moves lacking `flags.protect` still break through, by the same rule the authority applies. `data/mechanics-census.json` now reads **357 live and 357 probed**, three more than before this wire; Wide Guard's two existing probes are unchanged and green. The roster and the differential were not run and no roster row is claimed closed.

3.97.0 — THE DAMAGE FUNCTION WAS ONE ROLL MULTIPLIED BY N, AND FOUR MOVES PAID FOR IT. `dmgRange` ended `if(_hits>1) return {min: floor(roll(85)*_hits), max: floor(roll(100)*_hits)}` : everything a hit owns individually — its own base power, its own +2 , its own target — folded into a scalar. Triple Axel's `basePowerCallback` is `20 * move.hit` , so a flat 20 three times is **exactly half** the move (24 against 47); Dragon Darts carries `smartTarget` , so both darts landed on the aimed body and its partner took zero (−72/0 against −36/−34); Beat Up summed every ally's base power into one packet and lost three of the formula's four +2 s (24 against 28); and Fickle Beam's 30% double was applied as a flat ×1.3, giving 104 base power — a value the move never takes. The last is the 3.90.0 finding verbatim ("*the multi-hit count was the MEAN, and the pin never lands on a middle*") surviving in a second code path, with the comment above the line stating the averaging as a deliberate choice. The fix is a per-hit loop entered only where the artifact says base power depends on the hit index, so single-hit damage is unchanged **by construction** — and measured anyway: every move in the tag corpus, four real turns each, whole-board digests against the frozen release, **2,000 cells and 11 differences across exactly these four moves.** `data/mechanics-census.json` read 354 live and 354 probed at that release, four more than before this wire — a figure that has since moved on, and is quoted here as what the census previously held rather than what it holds now (3.98.0 took it to 357). No roster row is claimed closed; the roster and the differential were not run.

3.96.0 — THE ITEMS QUEUE WENT 6 TO 3, AND NOT ONE OF THE THREE WAS A MISSING MECHANIC. Each was a producer that could not name its member. `speedMult` was hardcoded to `name === 'choicescarf'`, so Iron Ball — which halves Speed through the identical `onModifySpe` handler — went untagged for 139 uses while the CONSUMER sat working and starved. `statMult` hardcoded four names of which **all four are banned in this format**, had no row in the artifact, and was read by nothing; `dmgRange` carried the matching hardcode for the same three banned items, three permanently-false conditions. Oran Berry heals a flat 10 HP rather than a fraction, and the derivation read only `maxhp/N`, so its amount came out null and the consumer refused — correctly, since a guessed heal is worse than none. CLAUDE.md's rule, *"match on tag shape, never on a name"*, is written for precisely this, and the rule immediately below `speedMult` in the same file records that lesson being learned for Life Orb while `speedMult` sat unfixed above it. **What this says about coverage:** a mechanic can be implemented, correct, and demonstrably live, and still be absent for a specific holder because the artifact never named them. The deliberate roster finds those because it stages every legal entity rather than the ones anyone thought to check.

3.95.0 — TWO READERS OF ONE FACT, AGAIN, AND THIS TIME IT WAS A QUARTER OF THE GATE. The damage differential's sole remaining disagreement across 150 comparisons was `chesnaught woodhammer -> mimikyu`: the authority reports `0-0`, this engine reported `120-130`. Showdown's Disguise returns false from `onDamage`, so the MOVE deals nothing and the `maxhp/8` that busts the disguise is the ABILITY's own damage, applied separately. The battle loop had this right and has since WIRE 136 — both engines land on the same HP, which is exactly why the Disguise model could be recorded as correct and the record be true. The DAMAGE CALCULATOR, which every board feature and every rollout leaf consults, knew nothing: measured against a control it returned **the same 120-142 with Disguise and with no ability at all**, the deliberate roster's own definition of an unwired knob. This is the `effMoveType / effWeatherOf` defect of 3.87.0 in a new place — one fact, two readers, one silent, each internally consistent so nothing ever failed. The fact is stated once now and both readers call it. **The hazard was in the fix rather than the finding:** the loop's damage comes FROM the calculator, so once the calculator correctly returned 0 the loop's `dmg > 0` guard would have been false precisely when the disguise was there to bust — the forme would never have broken. Caught before running. **Gate: 3 of 4 clauses failing to 2**, the differential clean at 0 of 150, all four re-measured under one release.

3.94.0 — THE MECHANIC WAS WORKING ON 20,000 USES, AND THAT IS EXACTLY WHY THE HOLE SURVIVED. Showdown carries the user's own stat change in **two different fields**. `self.boosts` covers Close Combat, Superpower, Draco Meteor, Overheat, Leaf Storm and Make It Rain — all six read FIRED-AND-BOARDS-MATCH in the deliberate roster, so "the user's own drop" looked closed. `selfBoost` is a separate field, the builder never read it, and the two moves in this format that use it — Clanging Scales (810 uses) and Scale Shot (199) — carried **no self-data at all**: Showdown drove the user to -1 then -2 Defence across two clicks while this engine left it at 0 both times. A sibling field name, not a missing mechanic, and no amount of re-reading the working path would have surfaced it. Roster moves **25 → 23** differ with exactly those two verdicts changed; census unmoved at 330 live; the damage table unmoved at 1728/1728 exact. **An alarm raised and then killed in the same pass, recorded because the killing is the useful part:** the `lowersUser` tag has no consumer anywhere in the engine across 13 moves and 22,277 uses, and that is *not* a hole — the engine applies these through the move row and the secondary path, never through that tag. A tag with no reader and a mechanic with no implementation are different claims, and only measurement separates them.

3.93.0 — SEVEN TRAPPING MOVES REPORTED THE IDENTICAL DIFFERENCE, WHICH IS HOW A FACT ANNOUNCES ITSELF AS ONE FACT. Bind, Fire Spin, Infestation, Sand Tomb, Snap Trap, Whirlpool and Wrap each read `showdown 4 / ours 3` then `3 / 2`. The artifact carried `partialTrap: { turns: '4-5' }`, typed by hand, and `'4-5'` is the folk quantity — how many turns of chip the trapped side feels. What the engines are compared on is Showdown's `partiallytrapped` **duration**, which starts at 5 and is

decremented in the Residual event **of the turn the trap lands**; this engine initialised from the already-decremented 4 and then ticked it again on that same turn. **The volatile-duration defect a third time** — Perish Song, then the family closed at 3.82.0, now this — surviving both because the counter lives in `_trap` rather than `_vo1`, outside either fix's reach. The shape is now read off Showdown's own condition rather than restated: duration from the condition, the `[5,6]` range from its `random(5,7)` callback, the Grip Claw branch, and the chip divisor from `onStart`'s ternary; it fails closed if the condition stops parsing. Red was demonstrated on the **frozen release the finding was measured under** (`3 · 2 · 1` against the authority's `4 · 3 · 2`), not asserted. Roster moves **32 → 25** differing with exactly seven verdicts changed and nothing else moved; census unmoved at 330 live. **The whole-game differential did not move — 65 of 107 games diverge on both releases** — which is stated because it is the measurement: a game stops at its first divergence and these moves rarely reach it.

3.92.0 — THE SWEEP THAT FOLLOWED THE GUARD FOUND FIVE MORE SITES, AND TWO WERE DEFECTS RATHER THAN COSMETICS. 3.91.0 caught `Tackle` — `isNonstandard: 'Past'` — padding one harness, and the obvious next question was whether it was alone. The `.item / .ability` assignment surface across 238 files is clean; move literals are not. Three sites were cosmetic, naming a nonexistent move in a slot that never acts. Two were not. `tests/test-priority-block.js` silenced three slots with Splash, which is not merely banned but ABSENT from the engine's move table — **so the silencing worked because the engine could not find the move, not because the move does nothing**, which is indistinguishable from working until it is not. And a guard in `tests/test-dead-volatile.js` admitted its subject only on `move.exists`, which is **true for a banned move**: the branch always ran, always on Thousand Arrows, which this format does not contain, and the else-branch that would have reported the gap was unreachable. Merely tightening that guard would have moved the hole rather than closing it, leaving the case untested — so the subject is derived from the format instead, and is now Smack Down. Every affected instrument re-runs green with no figure moved, which is the expected result: each was an inert slot or an unreachable branch. The value is that none of them can become live and wrong later.

3.91.0 — THE HARNESS COULD MEASURE A MECHANIC THE FORMAT DOES NOT CONTAIN, AND AGREEMENT ABOUT ONE IS WORTH NOTHING. `new Battle()` performs no validation, so a probe that assigns an ability or an item directly bypasses every rule in the format. Both engines will then agree about a Rocky Helmet in a format that bans it, and the row reads as a pass. `tests/probe_pair.js` now asks Showdown's own `TeamValidator` before it builds anything — the authority ADR-002 already names, reached through the instance `champions_sim.packTeam` already constructs, so the fact has one implementation. It catches strictly more than a ban-list check would: "*Meganium can't learn Flamethrower*" is a legal move on a species that cannot have it, which no `isNonstandard` test can see, and that exact set was hand-staged on 2026-08-08. **Two kinds of illegal are separated, because collapsing them would have refused every honest probe**: an entity the format does not contain is always fatal, while an entity this species merely cannot hold is a deliberate isolation in a controlled probe — `probe_pair` stamps one named quiet ability on every body precisely so the control does not vary with the species, and that ability is illegal on most of them. The second class is waivable with a written declaration; the first is not, and a self-test proves the declaration does not launder a ban. **The guard's first two findings were in the harness that hosts it**: `Tackle` is `isNonstandard: 'Past'` and every inert slot in the file carried it, and the five padding species this author named by hand included one that does not exist in this format. Both were names recalled instead of read, which is the same defect as the retraction one version below. Nothing downstream moved; this is an instrument, and every quarantined figure stays quarantined.

3.90.0 — A CLUSTER THAT DISAGREED IN BOTH DIRECTIONS AT ONCE, WHICH IS WHAT A COUNT ERROR LOOKS LIKE. Eleven multi-hit moves parted from the authority by small amounts, some high and some low, and the shape of that disagreement was the diagnosis: a per-hit rounding error cannot be both. The engine answered 3.1 hits — the mean of the 35/35/15/15 distribution over 2, 3, 4 and 5 —

to every question ever asked about the family, including the question a real turn asks. Read straight out of Showdown through `battle.choose`, so every hit runs, its `|-hitcount|` reports FIVE at the differential's top pin corner and TWO at the bottom and never a three: the authority samples a twenty-element table and the pin selects its first or last element. **The competing hypothesis, a per-hit floor, was ruled out with arithmetic rather than a preference:** the per-hit value is already an integer, so with an integer count `floor(v*n)` and `n*v` are the same number, and the line that computes it did not change. The count is drawn now, once per move use and after the accuracy steps, exactly where the authority draws it; the per-hit accuracy is rolled and breaks at the first miss rather than being discounted by a mean; and the count of REACTION events reads the same draw, so a Bullet Seed can no longer deal five hits of damage while setting off Weak Armor three times. Census 329 to 330 live, 0 missing; the roster's moves stage 40 to 32 disagreements with the eight multi-hit members accounting for all eight of the difference, abilities and items unmoved, the 150-row damage differential unmoved.

3.89.0 — A FAMILY THAT PRODUCED A WRONG ANSWER ON EVERY HIT, AND A HEAL FAMILY THAT HEALED 0.000 HP. 3.88.0 derived the condition for `buffsHolderOnHit` and said plainly that the engine did not read it; it does now. Applied unconditionally, eleven of the family's twelve members were wrong on every connecting hit — Anger Point maxed Attack off a NON-critical hit and did the same thing on a critical one (an unwired knob), Justified fired off Close Combat, Weak Armor off Dark Pulse. Stamina, 2,773 of the family's 2,972 uses, carries no condition and was correct throughout, which is precisely why nothing noticed the other eleven; it is now asserted on both sides of the crit die as the positive control. **The direction of the error is what distinguishes this from the pinch family of 3.85.0:** that one failed CLOSED, so the engine was merely missing a mechanic, while this one failed OPEN and was stating something false about the board. Separately, Synthesis, Moonlight, Morning Sun and Strength Sap (1,024 uses) resolved to a wasted turn healing nothing in every sky, and in sand the click was strictly worse than passing, because the residual still chipped the body that had spent its turn. The blocker was real and had expired: the tag said `heal: true`, a boolean in a fraction's clothing, and the artifact now carries the weather fractions and the target-stat reference. Census 326 to 329 live, 0 missing; the roster and the 150-row damage differential are unmoved.

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — TWO READERS OF ONE FACT, MEASURED: THE LOOP'S TYPE AUTHORITY AND THE DAMAGE CALCULATION DISAGREED ABOUT THE WEATHER. `effMoveType` resolved a weather-scaled move's type off the raw `field.weather`; `dmgRange` resolved it off `effWeatherOf`, which applies the private sky carried by the `privateWeather` tag. Under a private sun with a clear field the two part: the damage calculation returned Fire, 128-151, and the loop's stage-5 immunity gate refused the same click as Normal, so the damage dealt into a Ghost was 0. The official engine, played rather than cited (`gen9championsvgc2026regmb`, a real battle, Weather Ball into Gengar): 0/135 with `-immune` without the mega, 97/135 with it, and 97/135 under a public sun — the private sky must therefore give EXACTLY the public-sky number, not merely a non-zero one, because Showdown's `effectiveWeather()` feeds both `onModifyType` and `onModifyMove`. The probe asserts that equality. Census 325 to 326 live; the roster and the 1/150 damage

differential are unmoved; the paired whole-game differential is identical at 668 divergences of 1553 games in both arms, and the artifact names why — it lists this mechanic among the 47 census rows it declares unmeasurable.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — THE PINCH FAMILY FIRES, AND THE ENGINE'S REFUSAL WAS CORRECT THE WHOLE TIME. Four abilities — Blaze, Torrent, Overgrow, Swarm — carry 9,141 sheet uses and had never fired. The simulator refused them because their condition was recorded as the SENTENCE "only below 1/3 HP", and this project's rule is that a guessed threshold is worse than no wire. `engine/tag_dex.js` now derives the condition as a STRUCTURE out of the official engine's own handler, `{cond:'hpFraction', of:'self', cmp:'<=', num:1, den:3}`, and `condHolds` in `engine/medicham2-browser.js` evaluates it. The fraction is never collapsed to a float: the test is `hp × den ≤ maxhp × num` in integers, because `maxhp × (1/3)` is smaller than `maxhp / 3` and would refuse a body at exactly one third the boost it is owed. Anything the engine still cannot read refuses and is counted. New gate: `tests/test-pinch-family.js`, 61 rows against the official engine, red at 31 of 61 before the change.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all`-stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at `docs/` and `web/` but never at `app/`, which is the copy a visitor actually loads. Five withheld verdicts were being published from `app/` the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the

truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: PRE-CHANGE had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed from the differential and the roster, where a MISSING stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the CONTROL rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body Serious while the stored sheet beside it said Modest, and with every body flat AND Serious, 326 of 357 species in the format share a Speed with some other species — so the rig MANUFACTURED speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE — a Showdown open team sheet does not reveal them ("evs": null on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is CORRECT and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in clearVolatile, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is PERMANENTLY the thing it copied — so the two engines then choose replacements from benches that no longer describe the same

Pokemon, and worse, a Ditto can only ever transform ONCE PER BATTLE, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT. Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT. Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE INSTRUMENT RESOLVED A SWITCH BY TWO DIFFERENT KEYS AND FAILED SILENTLY BOTH WAYS. The driver names a bench member by Showdown's species id; the Showdown side looked it up by species id and the medicham side by the body's DISPLAY NAME. Those agree until a body is renamed — which this engine began doing the day before, when Disguise started renaming a busted Mimikyu, Zero to Hero started renaming Palafin, and Hunger Switch was queued to flip Morpeko every turn. After a rename the two keys part and that body can never be switched to again. Neither side raised anything: an unresolved lookup answered `pass` on both, so one engine could switch while the other stood still, producing a different board with no evidence attached. The key is now stamped at build time from the same expression the driver uses, and a miss is counted and printed beside the other declared gaps (0/0 over 120 games). This is an INSTRUMENT change rather than an engine one, so it alters what a measurement sees; it was also LATENT UNTIL THE FORME FIXES LANDED, and the deliberate-roster build would have walked into it.

WIRE 138-140 — THREE BOARD FAMILIES, AND A TARGETING MODEL THAT WAS WRONG WHENEVER ANYTHING MOVED (3.77.0). Aimed at the three largest surviving board-divergence families of the 1,530-game run at release 288aee2e3501 . **Speed Boost fired a turn early:** Showdown gates it on `activeTurns` , which is 0 on the turn a body switches in, and this engine's own comment said the gate "is not expressible here" — true of `_turnsOut` and untrue since WIRE 135 added `_newlySwitched` , a reason that was correct when written and stale when read. **A move targets a SLOT, not a Pokemon** (Will: *"we gotta target slots, not mons"*): `Battle#getTarget` resolves from `targetLoc` at execution time, and five of this engine's seven branches held the object they aimed at, so Charm and Parting Shot (10,535 uses: Charm 1,625 + Parting Shot 8,910, `data/tags.json` ; this read 7,184 when first written and the corpus has grown since) dropped stats on a body sitting on the BENCH. One shared reader now answers it everywhere, with `tracksTarget` (Snipe Shot, Stalwart) as the negative. **Ally Switch did not exist** — 202 uses resolving to a wasted turn — and it is the sharpest test of the slot rule, because both bodies stay on the field: before it, one unimplemented move parted TEN board fields at the end of a single turn. Each was RED on a staged board before its wire and IDENTICAL after; mega evolution, checked in the same pass, was already correct. Census 311 → 313 live, 0 missing; staged boards 18/18 identical and 18/18 breaks caught.

WIRE 133-137 — A SPEED TIE THAT HAS BEEN RESOLVED WRONGLY SINCE THE FIRST DAY, AND IT IS THE LIVE ENGINE (3.74.0). Measured on a staged pure tie under the differential's own primary pin: Showdown moved p2a first and medicham2 moved p1a first. The comparator was never the problem. `Array.prototype.sort` is STABLE, so a comparator returning 0 leaves the two in input order; `Battle#speedSort` is a SELECTION SORT whose swaps move UNTIED elements around, so the tied group's order when the shuffle finally sees it is not the input order, and no comparator can make a stable sort produce that permutation. ROADMAP #86 records that 91.4% of legal species share a base Speed with some other species, and `sortTurnOrder` is not an instrument — it orders every turn in every rollout and every live game. The fix reproduces the selection sort and resolves the residual group by the per-action uniform key already drawn, which is a uniform random permutation under real dice and the identity under a constant pinned die; **hardcoding the pinned answer was explicitly refused**, because it would have made the differential green on an engine that stayed wrong in play — the fitting-environment-versus-playing-environment error. Beside it, two board defects proven by staged comparison rather than by a probe (Zero to Hero's moment and Disguise's species), the switch-out trigger built as a CLASS after Will named it as one, and the last MISSING census row closed by enriching a tag that had described four different mechanics with one parameter. Census 298/299 → 310/310, missing 0 for the first time; `MEDFAILS.traceBodyOffField` 25 → 0.

ROADMAP #88 AND #91 — ONE PIN WAS ONE CORNER, AND A CLICK WAS COUNTED AS A TEST (3.73.0). Every die in the differential was pinned a single way, which bought determinism — any difference is a bug, no statistics — and paid for it in coverage nobody had priced. The speed tie always resolved the same direction, every move below 100 accuracy MISSED ON BOTH SIDES, and damage was always the maximum roll, which is the one roll where the crit's wrong position happened to come out right. Rock Slide had never connected in this instrument; under the new arms it misses in one and hits in another, and a crit lands in the bottom arm and not the top. The pin set is now a declared run parameter, digested into `mode` , and a before/after pair whose pins differ is REFUSED rather than reported. Separately, coverage credit moved from the CLICK to the OBSERVED EFFECT: the old rule incremented when an entity was clicked and never asked whether the move did anything, so Haze clicked into a board with no boosts on it — a no-op — marked Haze exercised and stopped the steering selecting it. Five rows were clicked-or-present and did nothing at all: `critDamageUp` , `preventsSwitch` , `privateWeather` , `clearsScreens` and `preTurnShield` . The old rule called all five covered. **THE BASELINE IS RESET: both changes alter which games get played, so no run after this is comparable with the turn-1 figure published at 3.71.0 or with** `data/state-ladder.json` . And an ENGINE defect fell out of the tie work, filed rather than fixed here: the two engines have disagreed about EVERY speed tie for the life of this instrument — the authority resolves a tie to the LATER

body in input order, `sortTurnOrder` draws one tie value per action from a constant scalar so the sort is stable and takes the EARLIER one. The instrument's own header claimed the pin made them agree by construction; that claim was false and was repeated as fact before it was checked. `sortTurnOrder` is the live engine, not instrument code.

A technical description of ABRA, a decision-support model family for competitive Pokémon

ROADMAP #92 — THE DAMAGE-STAGE CLASS. FOURTEEN MULTIPLIERS WERE APPLIED AT THE WRONG STAGE AND FIVE WERE ABSENT (3.73.0). Showdown applies each multiplier at a STAGE — a base power, a stat, or the final damage — folds every handler at that stage into ONE modifier, and spends it ONCE. This engine applied about a third of them a stage late, and separately: Black Glasses on the final damage where the authority puts it on base power reads 109 against 108. That one-point shape is why it survived every existing check — both engines "apply Black Glasses", so the census saw it LIVE, the interaction matrix compares a ratio between arms, and the damage differential allows a 12% midpoint band by design. LANDED: the 18 type items, Muscle Band, Wise Glasses, Technician, Tough Claws, Sharpness, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Sheer Force, Supreme Overlord, Expanding Force, Rising Voltage, Dry Skin and the -ate x1.2 into ONE base-power relay spent once; Thick Fat (73% wrong), Heatproof, Purifying Salt and Water Bubble (77%) into the STAT relay, because they modify a stat and not the damage; Helping Hand (wrong on 5 of 5 audited rows) and Friend Guard (21.4%) off the hit site and into the chains they belong in; Sniper out of the crit's plain multiply and into the final chain; and the rolled crit's POSITION into the damage range before the randomizer, where it was 46.5% wrong at the bottom roll and invisible at the top one every check pins. The four FIELD terrains were absent entirely — a Grassy-Terrain Earthquake was priced at DOUBLE the real number. New gate `tests/test-damage-stages.js` is **1,728 of 1,728 exact** against the authority across all sixteen damage rolls and both crit states, and was shown RED on two deliberate reversions before being trusted. `damageBoost` is still NOT wired as a class and the reason is a property of the tag: it carries neither the stage nor the condition, so wiring it would hand Blaze a permanent x1.5 on 5,808 sheets. Census 293/294 → 298/299 live.

ROADMAP #81 WIRE 12 — FIVE ENGINE DEFECTS OFF THE TURN-1 BOARD, TWO OF THEM MIS-DIAGNOSED BEFORE THEY WERE FIXED (3.71.0). The auras (Fairy, Dark, Aura Break) are wired FIELD-WIDE at the base-power stage — they multiply one type for every body on the field, the foe's moves included, and Aura Break INVERTS to x0.75 rather than cancelling; exact against the official engine on 12 of 12 staged arms. Baton Pass and Shed Tail switch for the first time (`passesState` had been derived and never consumed, so Baton Pass was a no-op turn and Shed Tail paid half its user's HP to stand still). Curse is two moves and the engine had neither. Perish Song counted from 3 instead of 4 and therefore fainted every affected body on both sides a full turn early, on 1,141 corpus uses — the KO itself had always fired. And ROADMAP #81 WIRE 10's measured board regression is one line: the Life Orb toll was being paid by a move that MISSED. **Two of the five briefed diagnoses were wrong** — the tagger was not testing `selfSwitch === true`, and the substitute doll was not confounded, it was a regression this project introduced at WIRE 7 on a misquoted source line. Census 281/282 → 293/294 live.

THE INSTRUMENT WAS MEASURING ANNOUNCEMENTS, AND THE HEADLINE IS NOW THE BOARD AT THE END OF TURN 1 (3.70.0). `engine/board_state.js` reads HP, status with its counters, items, all seven stat stages, aliveness, every field condition WITH ITS CLOCK and the persistent volatiles out of BOTH engines' live bodies at every turn boundary, after the whole residual phase. Read every figure from `data/state-ladder.json`. **The board at the end of turn 1 is identical in 56.0% of games at the pre-WIRE-1 baseline (1119/1998) and 66.9% at the top rung (1337/1998),** peaking at 69.3% at WIRE 9; whole-game board agreement went 6.4% -> 15.6% against a protocol number that read 1.8% -> 10.3%, so the wires were real and the protocol number overstated them. **WIRE 10 is a regression the**

protocol instrument scored as an improvement — 47 fewer clean turn-1 boards, and diffed per field it is one field, end-of-turn-1 HP wrong in 427 → 473 games. **41.0% of games whose narration parted inside turn 1 reached an identical board anyway.** The comparator proves itself first: 7 representation mappings red-demonstrated in both directions and 25 planted state divergences, each of which must be caught at the planted boundary and localised to the planted field — 25/25 on all fourteen arms.

A MORE CORRECT ENGINE DID NOT MAKE BETTER PREDICTIONS (3.69.0). The release ladder below records that ten fixes moved the differential's first-divergence depth from 13 to 19 protocol lines while the median completed turn never moved off 1. Which of the two readings — if either — predicts the thing the engine exists to serve had never been measured. It has now. **Read every figure from** `data/leaf-engine-contrast.json`. MILTANK's live in-game leaf (`rolloutWinProb`, `n=200`, `explore=1.0`, `foePolicy` uniform, horizon 60) was scored on **8,883 identical positions with identical per-position seeds** through two frozen engine releases that differ in **exactly one file**, `engine/medicham2-browser.js` — same weights, same board, same damage table, same tag file, same pinned Showdown commit; the generator refuses to run if that is not true. Paired Brier, WIRE 10 minus pre-WIRE-1, is **0.0000 with a 95% CI of [-0.0007, +0.0007]**, against a split-half noise floor of **0.000642** and a smallest detectable effect of **0.001013**. The interval is narrower than the effect the sample can resolve, so this is a **tight null and not an underpowered one**. McNemar over the 7,994 positions where both engines made a decisive call gives 37 discordant for WIRE 10 and 36 for the baseline ($p = 0.91$). **Neither depth metric predicts per-position leaf error.** Spearman rho between a position's first-divergence depth and that position's Brier: under the shipping engine **+0.0010 [-0.019, 0.022]** in lines and **-0.0000 [-0.021, 0.023]** in turns, against an MDE of 0.0298. Under the pre-WIRE-1 engine both are nominally significant, **both carry the wrong sign** (more correct simulation → larger leaf error, rho +0.031 and +0.029) and both sit at the detection threshold. The difference form — change in depth against change in error, in which every position-level confound constant across the two engines cancels — is **rho -0.0115 [-0.0307, +0.0082]** on 8,601 positions. The turn metric is **not** degenerate on this sample (13 distinct values, modal share 0.68), so the obvious escape is measured and rejected. A reversed-order control establishes that the null is not the ruler: re-reading the same positions with the coverage driver's history deliberately changed reproduces the depth at **rho 0.836 [0.825, 0.846]**. **What does limit the leaf is calibration, and it is a compression.** ECE **0.1514**; 88 points of predicted range map onto 13 points of observed range; when the leaf says 94% it wins 59%. Both engines remain decisively worse than a coin (paired Brier vs coin **+0.0325 [0.0281, 0.0372]**). Discrimination is 52.48% of 8,320 decisive calls in sample — against a 2.49-point split-half floor for a 2.48-point effect, so it is not an effect by this project's own rule — and **50.48%, $p = 0.70$, on the held-out newest fifth**. The engine fidelity gain is meanwhile real and replicates on this independent sample (games that never part 13 → 246, median divergence line 12 → 16, median completed turns 1 → 1). It simply does not reach the leaf. **Engine correctness is not the bottleneck; grinding the differential further cannot move the number the search is an argmax over.**

THE RELEASE LADDER, AND THE HONEST ANSWER IS THAT SIX FIXES DID NOT MOVE THE MEDIAN (3.68.0). `engine/wire_ladder.js` replays every frozen release of the 2026-08-06/07 wire night through the differential under one pinned census and one team pool, so all nine arms are mutually comparable rather than only adjacent — the defect that retracted the pairwise before/after in 3.62.1. **Read every figure from** `data/wire-ladder.json`. On 1,995 games per arm the median game parts after **one completed turn at every rung, unchanged**, and 22 of 1,995 games agree completely against 2 at the baseline. What did move is the DEPTH of the first divergence — mean 15.0 → 24.0 protocol lines, p90 30 → 57 — and per-rung effects that a pairwise comparison had misattributed: an intermediate cut that was never published as a wire outranks WIRE 1, whose pairwise before/after had absorbed it, and one unambiguously correct arithmetic fix moved **zero** of 1,995 games. The pre-WIRE-1 baseline was run first and last with eight arms between and reproduced exactly, so the ladder is the engine change and not the run.

THE DIFFERENTIAL HAS RUN, AND MEGAS ARE IN IT (3.62.2). `engine/game-differential.js` plays a real stored team through MEDICHAM and through the official Showdown engine, step for step, against a stamped frozen release. **Read every figure from** `data/game-differential.json`, **never from this sentence** — the first version of this paragraph quoted a run that a later one replaced within the day, which is the drift this whole document set keeps having to correct.

At the time of writing it reports every measured game diverging, with the median parting after a single completed turn. **Mega bodies are now tested** (ROADMAP #31): no stone is stripped from the measured arm, and every mega choice Showdown offered was taken by both engines.

Two limits travel with any rate this instrument prints and must never be separated from it. Nothing past the first turn is exercised, because a game stops at its first divergence. And both sides are built Serious / 0 EVs / 31 IVs so the two engines compute the same stat line before *and* after a forme change — **this tests RULES, not the spreads the ladder actually brings.**

This is a living document, updated in the same pass as any change to the code, together with the deck and the technical documentation. A prior conclusion is never silently rewritten; new information is added and what changed is stated. See `CHANGELOG.md`.

Abstract

ABRA is a decision-support model family for **Pokémon Champions VGC, Regulation M-B, best-of-one closed-sheet ladder**. It continuously ingests public battle replays from Pokémon Showdown and stores the durable facts of every game, then builds small, CPU-trainable models on that store. Its central empirical finding governs its design: **predicting the winner of a game from the two team sheets is near-impossible in this format — even a player-Elo model ties a coin**. ABRA therefore does not sell outcome prediction. It follows the recipe that worked in poker, Diplomacy, and sports analytics: *support decisions, don't predict outcomes*, and judge every model by a proper score against an honest baseline with a confidence interval. **As of 3.62.2 the headline metric is exploitability rather than win rate** (§0, ADR-003): VGC is formally an imperfect-information game, the only prior work in this exact format measures its own agents at approximately 100% exploitable despite beating a professional, and the thesis under test is that a per-turn re-solving agent is harder to exploit than a compiled policy — *unknown*, and stated as the experiment rather than the assumption. This paper states that thesis and its metric, the empirical ceiling, the data model, each model with its validated result (including two honest negatives), the mathematics, the limits, and the road to the in-battle engine (ALAKAZAM).

0. The thesis, and the metric that follows from it (3.62.2)

This project's headline metric is exploitability, not win rate. The change is recorded in ADR-003 and it is forced by a measurement somebody else made.

The field has been treating VGC as a chess problem or a pure-RL problem. It is a poker problem. Chess and Go are perfect-information games: there is one true state, both players see it, and minimax or MCTS over that state is sound. VGC is not. A player does not know which four of the opponent's six will be brought, their items, their abilities, or the fourth move on each set, and that hidden information is not noise to be averaged away — it is *strategically exploitable*. The correct solution concept is therefore a **Nash equilibrium in mixed strategies, not a single best move**, which is poker's situation verbatim and the reason RL+search methods that are sound in chess break in imperfect-information games (ReBeL, ref. 4). `docs/POKER-TO-POKEMON.md` works the correspondence through term by term and is honest about the three places it breaks — simultaneity, action-space and horizon scale, and the nature of chance.

That argument was made from theory and had no measurement behind it. **VGC-Bench supplies the measurement.** Angliss, Cui, Hu, Rahman and Stone (AAMAS 2026, ref. 5) trained behaviour cloning on 700,000+ human battle logs and fine-tuned with PPO under self-play, fictitious play and double oracle. In a single-team mirror match their agent **beat a World Championships competitor**. And in the same paper:

- *"In almost all cases, **all agents are approximately 100% exploitable**"* — measured by training a best-response policy against each agent.
- Their expert tester's feedback: *"although the agent is strong on initial play, it does have noticeable dips in performance in certain states. **After enough successive games, strong human players can adapt and beat the agent.**"*
- Against their *advanced* (not expert) tester the agent won **2 of 5**.

That is not a weakness of their execution. It is the predicted behaviour of a compiled policy — a fixed map from state to action — in an imperfect-information game. A best response can find and drill its blind spots, and so can a human given five games. Poker learned this over 2007–2021 and answered it with equilibrium mixing and continual re-solving (CFR, DeepStack, Libratus, ReBeL; refs. 1–4).

The thesis is therefore that a re-solving agent should be harder to exploit than a compiled one. A learned policy *recalls*; a search *recomputes*. A best-response exploiter attacks a fixed mapping, and a per-turn re-solve presents none. **Whether this survives simultaneity, stochasticity and a ~6-turn horizon is UNKNOWN — that is the experiment of this project, not its assumption.**

Two consequences for the model family. WOBBUFFET (§4, and `docs/MODELS.md`) moves from side-check to primary instrument, with VGC-Bench's ~100% as the published comparator. SLOWKING stops being "the preview solver" and becomes the shape of the whole agent.

And the honest state of that metric today is that we do not have one. `data/exploitability.json` is declared void; the 2026-07-26 figure was fitted on 17 features against the 58 we ship and the 2026-08-04 re-run had its defender refitted while it was running. Leading with a metric this project cannot currently produce is deliberate — it makes the gap a deliverable instead of a footnote.

0.1 Why the comparison is legitimate although the agents can never meet

VGC-Bench's public checkpoints are Regulation M-A; we are Regulation M-B, and their own paper shows policies do not transfer across team sets. A head-to-head is impossible. But **exploitability is intrinsic**: it is defined against a best response trained against *you*, in *your* format, so the two numbers live on the same scale without the two agents ever playing a game.

Two further points sharpen rather than weaken the frame:

- **VGC-Bench is open team sheets** — the same information setting as our Reg M-B best-of-three. They had *more* information than a closed-sheet agent and were still ~100% exploitable. The exploitability comes from holding a fixed policy, not from hidden teams.
- **Their dataset is not usable by us, and this project's code already said so.** Their Reg M-B holding is 4,167 games over 4 days in June 2026, and all 4,167 are already in our store as `data/games.ots.jsonl`, against our own 9,701 best-of-three games over 15 days. The 700,000 headline is Reg M-A, the previous regulation. An earlier claim in this project that their archive covered our format inferred coverage from a *filename* and is withdrawn; `docs/PRIOR-ART.md` §2 carries the correction.

0.2 What we are not claiming

- **Not that we will beat VGC-Bench.** Their agent beat a Worlds competitor; ours has never played a human.
- **Not that search is known to work here.** Metamon (RLC 2025, ref. 5) reached top 10% in singles with no search at all, and Future Sight AI removed its machine learning entirely after finding a structural

method beat it on both accuracy and speed. Both are live counter-evidence.

- **Not novelty on the format or the infrastructure.** VGC-Bench owns both, including the poke-env doubles support the field now uses. We are not first, and in our own format we are behind.

If the thesis fails, the instrumentation still stands. No project in `docs/PRIOR-ART.md` publishes a mechanics census that must be shown red before it counts, a step-level protocol differential against the official engine, ratchets on silent failure, or a record of what it retracted. If search loses, this remains the only account of what a hand-written VGC simulator gets wrong and how you would know — publishable precisely because everyone else avoided the problem by not having it.

0.3 The engine is justified if and only if search pays

VGC-Bench used real Showdown through poke-env and carried **no engine-correctness debt at all**, because behaviour cloning and PPO do not need a fast simulator: they need throughput at training time, not at decision time. We wrote MEDICHAM (§3) so that **per-turn re-solving is affordable**. That makes the engine work falsifiable rather than assumed, and it promotes one roadmap item to the status of a project gate. Supporting evidence that this is the real trade and not a rationalisation — every project that searches hits the engine-speed wall, and the pattern is clean:

project	searches?	engine	depth reached
VGC-Bench	no	real Showdown	n/a
Future Sight AI	yes	modified Showdown	~3 turns in 15 s on 16 cores
Foul Play	yes	built its own (poke-engine)	~10+ turns

The plan is four phases, and the fourth is a result rather than a defeat:

1	finish MEDICHAM	search needs an engine that is fast AND correct
2	GATE #62	does compute buy anything: untimed vs on-the-clock
3	if yes -> search, and measure EXPLOITABILITY against their ~100%	
4	if no -> adopt their recipe: BC + PPO self-play/FP/DO, open source, reproducible	

Phase 4 is cheap precisely because VGC-Bench made it so — the method is published, open-source and reproducible — and taking it would be a finding about VGC, not a failure of this project.

On compute. Cores help the search (it is CPU-bound and root-parallelisable); GPUs help behaviour cloning and PPO. MILTANK currently needs **26 s against a 20 s budget on one core of sixteen**, so sixteen cores fixes the clock today. But root parallelisation scales **sublinearly**, so cores convert a failed budget into a met one rather than a shallow search into a deep one. Buying cores does not buy depth, and the phase-2 gate is about depth.

1. The empirical ceiling (why the design is what it is)

On 600+ held-out real Champions games, a Bradley-Terry player-Elo model reaches a held-out log-loss of **0.687 against a coin's 0.693** — a real but negligible edge. **A 2026-07-25 re-measurement makes the ceiling lower still:** the previously published "higher-rated player wins 55.0%" was computed with a name-only bot filter that missed six high-volume accounts. Removing them gives **52.4%, 95% CI [49.9, 54.9]** — an interval containing a coin flip. A cloned-policy rollout engine (MEDICHAM) does *worse* than a coin as a raw win-predictor.

Re-measured 2026-08-04 on 6,886 clean games, against the leaves MILTANK actually calls rather than the `winProb2` entry point the earlier readings scored. Paired against a coin on identical turn-0 positions, the in-game leaf (`explore=1.0` , 200 rollouts, held-out n=1,378) loses by **Brier +0.0502, 95% CI [0.0371,**

0.0628], and the team-preview leaf (n=6,886) by **+0.0740 [0.0668, 0.0813]**; both also lose to player-Elo. The reliability curve is nearly flat — the in-game leaf's 90-100% bucket wins 53.6% and its 0-10% bucket wins 53.8% — and it names the winner on **50.99% of 1,314 decisive calls, 95% CI [48.3, 53.7]**, which is a coin. The preview leaf discriminates barely: 53.22% of 6,700 (CI [52.0, 54.4]), about 1.9 points above its own split-half noise floor.

This supersedes, and partly corrects, the earlier reading. The 2026-07-23 figure ("log-loss $\approx$ 1.2; picks the winner on $\sim$ 44% of decisive calls, i.e. systematically **inverted**") is retained here because a prior conclusion is never silently rewritten, but the inversion **does not replicate**: at twenty times the sample the same family of leaves sits slightly *above* chance, not below it. The 44% was a small-sample excursion. What replicates, and replicates decisively, is the **overconfidence**: the preview leaf puts 25.6% of its predictions into the two extreme buckets and is wrong there by about 40 points. Full curve, counts and intervals in `data/winrate-backtest.json`.

The conclusion is not "our models are weak." It is a property of the game: a two-player, zero-sum, **imperfect-information, simultaneous-move** game with a non-transitive metagame has an irreducible outcome-prediction ceiling from team sheets alone. This is the same reason expected-goals (xG) models in football predict *shot quality* rather than final scores. **Design consequence: stop predicting outcomes; support decisions.** Everything below serves that.

2. Data: store raw, analyse on top

ABRA reads Showdown's public replay API (`search.json?format=` , `search.json?user=` , `<id>.log`); it reads nothing private and creates no accounts (`SECURITY.md`). The extractor (`engine/durable-ingest.js` , `extract()`) turns one battle log into one durable record:

Field	Meaning
<code>id</code> , <code>date</code>	replay id and upload time
<code>p1</code> , <code>p2</code>	<code>{name, rating, bot}</code> per player
<code>six.p1/p2</code>	the revealed team of six
<code>brought.p1/p2</code>	the four actually brought
<code>lead.p1/p2</code>	the two led
<code>sets</code>	per species, the moves / item / ability the replay <i>revealed</i>
<code>turns</code>	per-turn events (moves, damage, faints, status, field)
<code>winner</code>	the winning name

The store is append-only JSON Lines keyed by replay id: idempotent, deduplicated at read time, and grown hourly by a GitHub Action. The **governing rule** is *store raw, analyse on top*: every filter (rating tier, humans-only, archetype, playstyle) is a re-computation over the store, never a re-pull. Changing how we segment games is free; the fetch is a one-time cost. About 2,600 public games/day are available, and the store grows $\sim$ 18k/week, so every model below sharpens on its own over time.

3. The validated foundation — exact damage (MEDICHAM)

The one component that is *not* a coin flip is the damage engine. MEDICHAM's Gen-9 doubles damage pipeline (`engine/medicham2-browser.js`) is validated against the Smogon damage calculator (the community ground-truth). This is gated in CI (`engine/validate_damage.js` $\rightarrow$ `data/damage-validation.json`). Every model that reasons about damage builds on this, and "will this move KO?" is a *winnable* prediction, unlike "who wins the game."

Read from `data/damage-validation.json` : **36 scenarios compared, within 5% on 100% of them, worst 0%**, at level 50 in gen 9.

This paragraph read "31 meta scenarios ... median error 0%, worst 3%" until 2026-08-22, and both halves were superseded rather than wrong when written. The harness gained scenarios (31 → 36) and the worst relative error fell to 0%; the artifact has said so since 2026-08-08 and the sentence did not move with it. The artifact's own caveat still governs what the agreement means: it is agreement on the DAMAGE FORMULA only, and says nothing about move selection, the accuracy model, or mechanics.

3.0 Why a hand-written engine exists at all, and the corrected speed figure (3.62.2)

ADR-001 decided this architecture on a benchmark of 29 against 3,401 battles/sec/core — a ratio of 117x — and that ratio does not reproduce. Re-measured on the same machine, both engines on the same four teams (derived from the store rather than typed), 8-second runs at a 60-turn cap:

	turns/sec	battles/sec
MEDICHAM	13,041	217
champions_sim	523	28
ratio	24.9x	7.7x

`turns/sec` **is the comparable unit** and `battles/sec` **is not**. The two engines were driven differently — MEDICHAM to its 60-turn cap, Showdown with `choose('default')` to a natural end — so a "battle" is not the same amount of work on the two sides, and the 7.7x is not like-for-like. The honest statement of the gap is **24.9x**. The July figures are retained above and in ADR-001 because a prior conclusion in this project is never silently rewritten, and a third reading exists that is neither: ROADMAP #61 measured MEDICHAM at 1,606 battles/sec. **Nothing ratchets engine speed**, which is how three readings of one quantity can differ by an order of magnitude with no test going red.

The architectural decision survives the correction, but its justification changes. A 24.9x gap still rules out live browser simulation, so ADR-001's conclusion stands. What no longer stands is "117x" as the reason. The reason is now the one §0.3 gives and it is falsifiable: **the engine work is justified if and only if search pays**, gated by ROADMAP #62.

3.1 The engine can now say WHAT it did, not only what state it reached (3.58.0)

The damage validation above, the 150-row differential and the five scripted whole-game comparisons all compare **outcomes**: a number, or a state after a turn. None of them can see an ordering, and none can say *which mechanism* produced a disagreement. Showdown's own protocol log can — it is a step-level trace already labelled with the mechanism behind each decision (`| -unboost|` is a stat drop, `| -enditem|` is an item being spent, the order of two `|move|` lines is turn order).

`engine/medicham2-browser.js` now emits that stream on request. The event set is **derived from Showdown's** `add()` **call sites** rather than transcribed (`engine/derive_protocol_events.js` → `data/protocol-events.json` , whose own `showdownEvents` , `emittedCount` , `notEmittedCount` and `partialCount` read 91 / 38 / 56 / 10), and two gates fail the run — claiming an event Showdown never emits, or leaving one unexplained. The scan reads this **format's** overrides, not the generic protocol: Champions emits `| -supereffective|POKEMON|N` where the base engine emits two fields.

It changes no mechanic, and two things it found on its first night are recorded because they are about **what the existing instruments cannot see**:

- 1. **The damage differential is an endpoint comparison.** It calls the reference at `roll=0` and `roll=15` against MEDICHAM's `min` and `max` . In between, MEDICHAM interpolates linearly over an 11-integer range and samples it uniformly; Showdown floors sixteen base values separately. 149/150 endpoint

agreement is compatible with every interior roll being off by one or two, and with every roll's *probability* being wrong. This is a limitation of the measurement, stated; it is not a claim that the damage is wrong.

2. **Order within a hit differs and end-of-turn state does not.** MEDICHAM resolves the knock-off, the resist berry and the contact punish before subtracting the target's HP. The whole-game state comparison agrees on every turn of all five scripted games; the trace does not agree on the order.

Neither is fixed here. Changing how a damage roll is drawn moves every seeded run in the repository.

Finding 1's limitation was closed on 2026-08-22, and the closure is recorded rather than written over the sentence above. The differential no longer speaks only for the two corners. `data/engine-diff.json` now reads **6000 requested, 6000 compared, 5995 agreed, 5 disagreed**, `band_missing` **0**. The artifact states the rule the sweep enforces in its own `band_why` field: *"THREE SAMPLED POINTS OF A SIXTEEN-INDEX BAND CANNOT SPEAK FOR THE THIRTEEN THEY NEVER SAMPLE"* — and its `arms_why` field records the separate demonstration that a midpoint cannot see a range wrong by the same amount at both ends, because `--plant spread` leaves `disagreed` at its unplanted value while both corner arms light up. So the paragraph above is now history: it was a true statement about an endpoint-only instrument, and the instrument changed.

Every probability ships a **proper score** (log-loss and/or Brier), a **confidence interval** (clustered by game where states within a game are correlated), and an **honest baseline**, persisted to JSON and gated in CI.

4.1 GURU — meta / matchup matrix (descriptive)

From REAL outcomes, `engine/guru.py` builds an archetype × archetype matchup matrix (K is chosen from the data; see `data/archetypes.json`) over the generated game count in `data/live.js` (hardcoded sizes are retracted, S13) — games, each cell a win-rate with a **Wilson score interval**. GURU is *descriptive*: its own predictive test shows per-game winner prediction from the matrix ties a coin (log-loss 0.7122 vs 0.6931), exactly as \$1 predicts. Its value is honest matchup structure with error bars, and it is the real (not simulated) payoff matrix that SLOWKING solves. Output: `data/guru-matchups.json`, `data/guru.js`.

4.2 XATU — opponent belief (modest, useful)

`engine/xatu.py` learns, per species, the set (item/ability/moves) usually run, and predicts the opponent's next move from state. On held-out human moves the behaviour-clone reaches **top-1 35.9% (CI 35.2–36.5)**, **top-3 71.6%**, cross-entropy 2.27 nats — beating a species-agnostic baseline (4.54) and uniform-over-moveset (2.91). A modest but real signal; human move choice has genuine entropy. Output: `data/xatu.json`, `data/xatu.js`; harness `engine/eval_policy.py` → `data/policy-eval.json`.

4.3 PORY — mid-game win probability (RETRACTED as a value net; it is material arithmetic)

The pivot's proof. `engine/pory.py` reconstructs per-turn board state (mons alive out of four, mean active HP, turn) and fits a logistic value net. Held-out, clustered by game: **log-loss 0.6236** 95% CI [0.6070, 0.6387] vs coin 0.693, **beating a material-sign heuristic**, calibrated to ECE 1.6%, **CI [0.548, 0.583]****. The *live board is predictable even though the pre-game sheets are not* — the thesis, demonstrated. PORY is wired into KADABRA as a per-turn "you're at X%". Output: `data/pory.js`; report `data/pory-eval.json`.

4.4 CHOMP-EV — do CHOMP's brings beat humans'? (honest NULL)

The winnable team-preview test. For each held-out game (both full sixes, both actual brings, the winner), `engine/chomp_ev.js` ranks each side's *actual* bring among all 15 candidate brings by CHOMP's exact-damage coverage, and asks whether that quality signal tracks who won. On **1,205 games**: CHOMP's bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), ties an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). It is **robust to forfeits** (0.505; a forfeit is usually a concession from a losing position, and dropping

all forfeits does not change the result), and a measured **selection audit** shows the required "all four revealed" filter is a mild bias (eval 6.5 turns / 1280 rating vs 6.08 / 1267 excluded) that, if anything, *favours* CHOMP — making the null conservative. A **belief-weighted** variant (coverage vs the opponent's likely-4) also ties the coin (0.6924). Interpretation: the bring decision sits at the same near-coin ceiling as pre-game prediction; CHOMP's damage math stays validated and useful as a calculator, but "CHOMP builds better brings" is not yet empirically supported. This negative is a guardrail: it stops optimising a bring metric that carries no held-out winning signal. Report `data/chomp-ev.json`; test `tests/test-chomp-ev.js`.

Multiplicity, corrected 2026-07-31. The fit reports a 95% interval for all 56 features, so at alpha 0.05 about **2.8 of them clear zero by chance alone**. The family is **every feature in the shipped fit**, because every one is reported to the reader — choosing a smaller family after seeing which are large is the practice the correction exists to prevent. Uncorrected, **53** clear zero. Under **Benjamini–Hochberg** (FDR, 1995) **53** survive; under **Bonferroni** (FWER) **49**. Nothing significant uncorrected fails the FDR correction, so the headline count is not an artefact of having looked at 56. Computed by `engine/weight_multiplicity.js` → `data/weight-multiplicity.json`. **This says which weights are distinguishable from zero. It says nothing about whether an imitation-fitted weight is evidence about WINNING** — a separate and larger question this project has measured going the other way.

A phrasing the filter itself mandates. `require_full_bring` conditions on game length: measured 2026-07-31, the games it keeps are **1.71x longer** on average (7.4 vs 4.3 mean turns; 19,589 kept vs 8,713 dropped). Every bring statistic in this project is therefore "*the bring, among games long enough to show it*", which is not the same as "the bring". `data/quality-filter.json` states this at the point of filtering and requires it to be said downstream; this is that.

4.5 SLOWKING — team-preview Nash and the playstyle cycle (suggestive)

`engine/slowking_preview.py` solves a matchup matrix to a mixed-strategy equilibrium and grades it by **exploitability** (the worst-case win-edge a best pure counter extracts; lower is better; Nash ≈ 0), against greedy "single best deck" and uniform baselines, with a bootstrap CI that propagates matchup-count uncertainty (Beta resampling). Over GURU's 13 species-archetypes the equilibrium is far less exploitable than uniform (Nash ≈ 0 vs 0.109), but greedy $\approx$ Nash because this meta is currently near-transitive (a dominant deck). A **playstyle** re-analysis (`engine/playstyle.js` classifies each team as TrickRoom / Rain / Sun / Sand / Snow / Setup / PerishTrap / TailwindOffense / FakeOutBalance / Stall / HyperOffense) surfaces a non-transitive cycle — **TrickRoom** → **HyperOffense** → **Sand** → **TrickRoom** — with a point exploitability gap of ~ 0.073 for greedy; the equilibrium now correctly leads with Sun ($\sim 31\%$), since Reg M-B Charizard is Mega-Y (Drought) and is classified as a Sun setter. **Honest caveat:** each cycle leg rests on only 13–18 games (win rates 73% / 71% / 67%) with 95% CIs that cross 50%, so the cycle is a **suggestive pattern, not a settled fact**; it will sharpen as the store grows. Where matchups *are* well-sampled they tend to run flat against intuition — **Rain vs Sun is 51% (n=236)** and **Tailwind vs no-Tailwind is 47% (n=756)**, both statistical coin-flips. Reports `data/slowking-eval.json`, `data/slowking-playstyle-eval.json`; test `tests/test-slowking.py`.

5. Mathematics

Wilson score interval (used for every matchup rate) for w wins in n games, $z = 1.96$: $(\hat{p} + z^2/2n \pm z \cdot \sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2}) / (1 + z^2/n)$, with $\hat{p} = w/n$. It is well-behaved at small n and near 0/1, unlike the normal approximation.

Value net. Features $x = [\text{alive_diff}, \text{hp_diff}, \text{my_alive}, \text{foe_alive}, \text{turn}/10]$ are standardised by train mean/std; $P(\text{win}) = \sigma(w \cdot z + b)$. Graded by held-out **log-loss** $-(y \cdot \ln p + (1-y) \cdot \ln(1-p))$ and **Brier** $(p-y)^2$; the coin scores $\ln 2 = 0.6931$ and 0.25 respectively.

Discrete choice — the scoring bot's policy (v3.28.0). A player facing a turn chooses one of the legal (move, target) pairs. Writing x_j for the attributes of alternative j — type effectiveness against that specific target, base power, whether the move is already dead on the board, and the behaviour clone's $P(\text{move} \mid \text{species})$ — the conditional logit model (McFadden 1974) is

$$P(\text{pick } j) = \exp(w \cdot x_j) / \sum_k \exp(w \cdot x_k),$$

with w estimated by maximising $\sum_i [w \cdot x_{\{i, \text{chosen}\}} - \ln \sum_k \exp(w \cdot x_{\{i, k\}})]$ over **146,910** real human decisions from **6,091** clean open-sheet games (117,824 train / 29,086 held out), with a feature vector that has grown from 12 to **53**. The weights are **estimated, never written down**, and the realism report is never consulted during fitting — it is held back as the out-of-sample check, because a diagnostic stops being evidence once it becomes the objective.

Held out **by game** (decisions within a game are correlated — the same clustering argument as the CIs below): logL/decision **-1.6006** and top-1 **33.6%**, against the behaviour clone alone at $-1.9302 / 27.1\%$ and uniform at $-1.7627 / 24.1\%$. Open-sheet games are used because they are the only corpus in which the **choice set** is known rather than guessed: a normal replay reveals only the moves that were *used*, so alternatives reconstructed from revelation are biased by revelation.

The model's known limitation is **independence of irrelevant alternatives**: logit implies the odds between two options are unaffected by what else is on the menu, which fails for close substitutes (the red-bus/blue-bus problem). A set carrying two moves of the same type is exactly that case. Nested or mixed logit is the remedy and neither is implemented, so the fitted probabilities are a good ranking and only an approximate distribution.

Equilibrium and exploitability. Each preview is a two-player zero-sum matrix game on an antisymmetric edge matrix $M[i, j] = (p(i > j) - p(j > i)) / 2$. Regret matching (Hart & Mas-Colell) converges to an ϵ -Nash. For a strategy x , **exploitability** $= -\min_j (x \cdot M[:, j])$ — the worst-case loss to a best response; the Nash value is 0, so a Nash strategy scores ≈ 0 and a predictable single-deck strategy is punished.

Confidence intervals. Because per-turn states within a game are correlated, CIs are **bootstrapped by resampling games** (clustered), not states. Matchup-matrix uncertainty is propagated by **Beta($n \cdot p + 1$, $n \cdot (1 - p) + 1$) resampling** of each cell before re-solving.

Future rating math. For any descriptive meta-rating we will use an **intransitivity-capable** class (blade-chest / low-rank bilinear, Chen & Joachims 2016; or Nash-averaging, Balduzzi et al. 2018) and a **Helmholtz–Hodge / HodgeRank** decomposition (Jiang, Lim, Yao & Ye 2011) to split the matchup flow into a transitive ranking plus a cyclic (rock-paper-scissors) component — the correct tool for "which cores beat which" and for quantifying how cyclic the meta really is.

6. Limitations and honest ceilings

1. **The game-winner ceiling is permanent** (Elo $\approx$ coin). SLOWKING/ALAKAZAM are judged on decision quality and self-play/ladder win-rate, never on match-outcome prediction.
2. **Revealed sets are partial** (a mon that never attacked reveals no moves); belief is a lower bound.
3. **Small samples in the meta layer.** Playstyle and core matchups are thin; those results are suggestive until the store grows.
4. **Policy is the residual GIGO — and in 3.28.0 the binding constraint is the OBJECTIVE, not the knowledge.** The damage is validated. The policy now runs a real damage calculation and does decide switches — both were listed here as missing and both became false, and they are corrected rather than quietly dropped. What remains: **one ply**, no model of the opponent's move, no search.

The sharper limitation is measured. Over 2026-07-30, **four separate feature additions produced four nulls**, while **two changes to the objective produced two large wins** — greedy action selection at +12 points (79.7% of decisive pairs) and self-play policy improvement at 55.9%. An overdispersion check across teams (~1.00, against 1.169 for a known real effect) rules out the obvious confound, so the nulls are genuine. Adding knowledge to an imitation-fitted policy has stopped paying.

RECONCILED 2026-07-31. *That 55.9% was measured on the 53-feature vector with switching OFF. Repeating the experiment on the 56-feature vector with switching ON gives 48.1% [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.*

The cleanest demonstration is the pair-scoring layer (DODUO), which is **built, wired, controlled and measured, and loses at 42.0%** [39.9, 44.3] over 1,934 seed-paired games against its own zeroed control. Its fit prices "use a spread move beside my own ally that does not hurt it" at **-5.054** — a statement that humans rarely click it, not that it is bad. Refitting those weights for *winning* rather than *resemblance* is untested and is the project's top open question.

CORRECTED 2026-08-01, and this paragraph should no longer be cited as it stands. *The -5.054 was not a statement about human preference. `fit_joint.js` matched a human's click by requiring the candidate's target to match, and a spread move is built with no target because it is not aimed — so **no spread click could ever match**. Spread moves are 14.94% of all human move clicks and 99.7% of them were thrown away; the fit used 24,997 of 82,483 joint turns, and the discarded 70% was exactly the turns containing the play the feature describes. Refitted, the weight is **+0.863**, and the corrected vector beats the shipped one at **66.7%** and **65.9%** of decisive pairs on two disjoint seed blocks. DODUO's 42.0% was measured on the contaminated vector and does not describe the current one. The imitation-versus-winning argument stands on its other evidence — greedy action selection is worth about 12 points — but not on this example.*

A separate class of defect, worth naming because it is not a modelling disagreement: a fact reaching one consumer and not the next. Priority blocking lived in the tag artifact and never reached the simulator, so **Sucker Punch beat a Farigiraf in every rollout ever run**. A switch-in's own ability never reached the code that chooses the switch — over 40,001 matchups, declaring Intimidate, Drizzle or Drought moved the feature vector in **zero** of them against a control's 2,754. And every mega forme carried a null ability, an empty moveset and no item, so **26.0% of the format's usage scored as threatening nothing**. All fixed; a gate (`engine/artifact_audit.js`) now compares derived artifacts against their sources, because nothing had.

Every `build_lab` win rate on record was measured against the older board-blind pilot and none has been re-run, so all of them remain provisional.

1. **Champions rule specifics** (sleep/paralysis edge cases) are flagged, not yet fully modelled.
2. **A result that does not record its own configuration is not reproducible, and three of the four rollout gates were in that state.** This is a methodological limitation, added 3.33.0, and it cost a published result. The R1 gate reported a PASS; recomputed from the only committed evidence it is **UNDECIDED**. No number was falsified. The row dump recorded `{gid, turn, p, mpy, y, aliveDiff, hpDiff}` and no sample size, no exploration rate and no build digest, so a dump taken at `explore=0` and a dump taken at `explore=1` were byte-compatible while the two configurations do not give the same answer. Only the surviving calibration shape distinguished them, in hindsight.

THE RATES AND INTERVALS THIS ITEM USED TO QUOTE ARE WITHHELD, 2026-08-22. R1, R2 and R3 read artifacts downstream of MEDICHAM, and `node engine/status.js` names each one QUARANTINED. The limitation being described is about the CONFIGURATION RECORD and survives without them; a quarantined figure printed with a caveat beside it is the failure this section is about, one level up.

Auditing the other rungs against the same standard produced two further findings. **The R3 divergence gate publishes a rate and records no control.** Its own script computes the quantity that makes a divergence rate mean anything — the same search on a different seed disagreeing with *itself*, whose true value is 0 by construction — writes it to standard output, and does not store it; the script's verdict branches on that comparison, so the artifact cannot state which branch its own run took. At a rollout budget of N=20 that floor measured *higher* than the divergence it was meant to validate. **The R2 cost gate timed a leaf the system does not run**, inheriting library defaults of `explore=0` and a 20-turn horizon while the deployed leaf uses `explore=1.0` at 60 turns.

The general statement is that a search is worth exactly what its leaf is worth, and a leaf measurement is worth exactly what its configuration record is worth. Every gate artifact now carries a sidecar (`engine/run_stamp.js`) recording the budget, the exploration rate, the horizon, content digests of every source the gate reads, the commit, and whether the working tree was dirty at the time. Artifacts predating the standard carry a stamp reconstructed from the commit that contained them, labelled as inferred rather than observed on every field.

1. **The headline metric has no current value, and that is the largest limitation in this paper** (added 3.62.2). §0 makes exploitability the number this project is judged on, and `data/exploitability.json` is declared void: the 2026-07-26 figure was fitted on 17 features against the 58 shipped, on an engine 25 wire-fixes old and before the quality filter existed, and the 2026-08-04 re-run had `data/policy-weights.json` — the defender itself — refitted at 22:15:24 UTC while it was running. **So the comparison with VGC-Bench's ~100% is a comparison we have set up and not yet made.** Producing one figure requires training a best response against a frozen agent, which is expensive, and it requires the frozen-release discipline to hold for the whole run — the 2026-08-04 void was an exploitability run, so this is a demonstrated failure mode rather than a hypothetical one.
1. **Two speed readings of the same engine differ by an order of magnitude and nothing caught it** (added 3.62.2, §3.0). 3,401, 1,606 and 13,041 are three measurements of MEDICHAM's throughput taken over two weeks; the first two are battles/sec and the third is turns/sec, and no ratchet, test or artifact compares any of them. A project whose central architectural decision rests on a speed ratio should measure that ratio the way it measures a win rate. It does not, yet.

7. The road to ALAKAZAM

ALAKAZAM is the in-battle capstone, built last on the inputs above. Given a live position it will output the win-%-optimal move (a mixed strategy) and its value by: (1) a **belief** over the opponent's hidden sets (XATU), updated by a Bayesian filter; (2) **depth-limited search** over the validated damage engine, solving each simultaneous turn as a **matrix game** (regret matching — this removes the speed bias that inverted the greedy engine); (3) a **learned value** at the leaves (PORY, grown to an NNUE-style net); (4) **human-anchoring** (KL-regularised to the behaviour-clone) so it stays strong and unexploitable. Inference is light (CPU / Web Worker / WASM); the strongest version needs offline RL on millions of human + self-play games and a rented cloud GPU. It is judged on decision quality and self-play/ladder win-rate with CIs — never on predicting the winner. A self-play data engine (MEW) is the pacing item toward the millions of games that path needs.

Sequenced by the four phases (3.62.2, §0.3). ALAKAZAM as described above is phase 3. It is reached only through phase 1 (MEDICHAM complete — a search needs an engine that is both fast and correct) and phase 2 (the gate: MILTANK untimed against MILTANK on the clock, ROADMAP #62, which decides whether compute buys anything at all). If phase 2 says no, phase 4 replaces this road with VGC-Bench's: behaviour cloning plus PPO under self-play, fictitious play and double oracle. That branch is approved in advance and is a result about the game, not a defeat — and it is cheap because the method is published and reproducible.

8. References

1. Zinkevich et al., *Regret Minimization in Games with Incomplete Information* (CFR), 2007.
2. Lanctot et al., *Monte Carlo Sampling for Regret Minimization* (MCCFR), 2009.
3. Moravčík et al., *DeepStack*, Science 2017. · Brown & Sandholm, *Libratus*, Science 2018.
4. Brown et al., *Combining Deep RL and Search* (ReBeL), NeurIPS 2020. · Schmid et al., *Player of Games*, 2021.
5. Angliss, Cui, Hu, Rahman & Stone, *VGC-Bench: A Benchmark and Strategy Suite for Competitive Pokémon Doubles Battling*, AAMAS 2026, [arXiv 2506.10326](#) — the only published work in this exact format; the source of the ~100%-exploitable finding and of the professional-beating result quoted in §0. · Grigsby, Xie, Sasek, Zheng & Zhu, *Metamon* (offline RL + large sequence models, no search), RLC 2025, [arXiv 2504.04395](#). · Full survey of the field, with what each project implies for this one: [docs/PRIOR-ART.md](#).
6. Perolat et al., *DeepNash / R-NaD* (Stratego), Science 2022. · Vinyals et al., *AlphaStar*, 2019.
7. Meta FAIR, *CICERO / piKL* (human-regularised RL, Diplomacy), Science 2022.
8. Chen & Joachims, *Modeling Intransitivity in Matchup Data* (blade-chest), WSDM 2016. · Balduzzi et al., *Re-evaluating Evaluation* (Nash-averaging), NeurIPS 2018.
9. Jiang, Lim, Yao & Ye, *Statistical Ranking and Combinatorial Hodge Theory* (HodgeRank), 2011.
10. Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*, JASA 1927.
11. McFadden, *Conditional Logit Analysis of Qualitative Choice Behavior*, in Zarembka (ed.), **Frontiers in Econometrics**, Academic Press 1974 — the discrete-choice model the scoring bot's policy is fitted with (§5).
12. the Smogon damage calculator — community damage ground-truth. · Pokémon Showdown replay API.

Companion documents. [Slide deck](#) · [Technical documentation](#) · [Model ledger](#) · [Changelog](#)

The role family: multi-label composition, WAR, and emergent roles (v2.6.0)

Motivation

The earlier playstyle model assigned each team exactly one archetype. This is a **multi-class** framing of a **multi-label** object: a real team is Sun *and* Tailwind *and* Fake Out at once. Forcing one label discards most of the information and shatters the data into archetype×archetype cells of $n \approx 11-18$, which is why those matchup numbers were untrustworthy. The literature is explicit: multi-label classification (Tsoumakas & Katakis 2007), team-as-mixture-of-latent-roles (topic models; Blei-Ng-Jordan 2003), and latent roles beating raw identity for outcome prediction in team sports (arXiv 2304.08272).

Role tagging (leak-free, data-earned)

We define 26 functional roles. A **species earns a role from data** — it is credited once it is observed performing the role (≥ 2 times) across the store. Multi-effect moves carry several *factual* roles (Matcha Gotcha = special+heal+status; Body Press = wall+attack; Fake Out = tempo, not attacker). Role *presence* is binary; graded *strength* is deliberately **not** hand-set (asserting weights violates the project's measurement standard). A team's role vector is built from the **team-preview six**, which are public in every closed-sheet game, so the representation is uncensored and non-leaking.

Each ordered role pair (a, b) aggregates outcomes across every game where one side had a and the other had b, with a Wilson score interval. Because roles co-occur, each game contributes to many cells, so the **median cell rises from $n \approx 15$ to $n = 20$ across 1,051 cells (measured 2026-07-25 on 1,061 quality-filtered games)**. The figure of 7,971 published in v2.6.0 was retracted in 2.7.0 as an artifact of over-tagging (19.6 of 26 roles per team); it has since gone $7,971 \rightarrow 95 \rightarrow \sim 50 \rightarrow 20$ as the taxonomy sharpened and the games were filtered — the structural fix. Empirically, however, a logistic model on the preview role-difference vector predicts the winner at held-out log-loss 0.694 vs a coin's 0.693: **roles describe and attribute, they do not predict**. The per-role coefficients are read as **win-credit per role**; KO-credit per species is measured directly from the turn log.

WAR — Wins Above Replacement (species RAPM)

To attribute wins to individual Pokémon while controlling for teammates and opponents, we use basketball's **Regularized Adjusted Plus-Minus**. With one row per game, label $y = 1$ if p1 won and features $x_s = 1[s \in p1 \text{ six}] - 1[s \in p2 \text{ six}]$, a ridge logistic regression yields β_s , species s's adjusted win contribution. Ridge shrinks rare species toward zero. With replacement β at the 20th percentile and the logistic slope $1/4$ at $p = 0.5$,

$$\text{WAR}_s = 0.25 \cdot (\beta_s - \beta_{\text{replacement}}) \cdot (\text{games } s \text{ appeared}).$$

Held-out, the species model reached log-loss 0.6875 against a coin's 0.6931 — a result now **withdrawn 2026-07-25** — that figure was measured on the UNFILTERED store; on quality-filtered games WAR scores 0.7048 against a coin's 0.6931 (accuracy 0.502). The apparent signal was four bot accounts playing one team in 1,446 games. It does not beat a coin: *which specific species* you bring at preview carries a small real signal that roles and raw sheets do not. Leaders are Basculegion, Kingambit, Sylveon; trailers are negative. Effect sizes are small and magnitudes ridge-shrunk — reported as an exploratory ordering, not settled wins.

Emergent roles by NMF

Rather than hand-declaring roles, we factorize the data with **Non-negative Matrix Factorization** (Lee & Seung 1999): $X \approx WH$ with $W, H \geq 0$, so each team is a non-negative **blend** of latent roles and each role is a recipe over features. Two cuts: (1) the team×move usage matrix (usage-weighted, which down-weights the closed-sheet censoring skew) recovers **offensive cores** but is dominated by attacking moves (relative reconstruction error 0.79); (2) the team×role matrix recovers **six clean archetypes** (error 0.53): Intimidate+Fake-Out control, physical offense, special offense+sustain, bulky wall+screens+ redirection, Tailwind+Encore, priority. A move's loading on a role is **learned, not typed** — this is the principled source of graded primary/secondary strength (Label Distribution Learning, Geng 2016). The rank and the human names are the only non-data choices. Reconstruction error is **not** comparable across weightings; the correct model-selection criterion is **topic coherence** (Mimno et al. 2011), noted as the next refinement.

Honest limits

Preview-composition signal is small; role-level winner-prediction ties a coin and WAR barely clears it. Role tags are a censored lower bound on capability (closed sheets reveal only used moves). NMF factors are soft and attacker-dominated at the move level. None of these is hidden; each is reported with its baseline.

The coverage job lands, and the plan that drives it is amended (3.40.0)

Three results, each read from its artifact.

The engine. Wires 82–89 landed: the pre-turn shield class (Focus Punch / Beak Blast), the variable-power family, per-hit reactors, priority blocking across every move kind, Memento, drain-before-contact-toll order, the Steel Roller terrain gate, and secondary chances read from the FORMAT's rulebook with a drift counter.

`data/mechanics-census.json` moved 167 → **181 live of 186 probed, 5 missing with reasons**; the interaction matrix moved 68 disagreements → **13** (1,012 live carrier × reactor cases, 999 agree, 98.7%); the Showdown damage differential stands at 1/150, and the one row is a documented harness-layer artifact (Disguise), not an engine defect. Every new probe was demonstrated red against a deliberately broken in-memory engine before its green was believed. Shell Trap, flagged as entirely untagged, is `isNonstandard: 'Past'` — banned in this format; the missing tag is the format door working.

The two-rulebook question, measured before it was architected. `data/tags.json` and `CHOMP/data/move-effects.json` state overlapping move facts. Compared field by field (`data/rulebook-collision.json` , a ratchet that may fall and never rise): 151 comparable facts, 149 agree, **2 clashes** — and the live one was Iron Head's flinch, where the tags copy carried the Champions format's 20%, the generic copy carried 30%, and the engine read the wrong one. Wire 89 closes it at the consumer. The unified-generator layer of the coverage plan is therefore insurance rather than urgency; the real exposure is the facts the comparison cannot yet reach — **27 tag-only and 166 fx-only**, read from that artifact's own `not_compared` block rather than summed in prose. *(This stated their SUM until 3.70.0. That sum appears in no artifact; it passed the currency check only because the same numeral happened to occur somewhere inside `data/tags.json` , and it went red the moment that file was regenerated. A figure that traces by coincidence does not trace.)*

The fitting gap below is now half closed. The sheet-channel section that follows reported that the fit discarded the ability and moves the live player sees; the decision it asked for was made (open team sheets always, closed sheets deferred), and the single-move layer (MAG) is refitted on all four channels: 232,815 usable decisions of 241,927 seen at 3.42.0 (231,722 at the 3.40.0 fit, before the click-censoring pass removed 1,336 actions that were not clicks), with a point-of-use counter showing the declared channels reached the board on 99.67% of scored decisions — an environment match stated by measurement, not by diff-reading. The pre-refit weights are preserved and the two-channel incumbent is frozen as a release (`d3d04b669e18`) for the pending paired held-out comparison against the 0.192-point noise floor. The joint (pair) layer is **not yet refitted**; until it is, the pair layer still prices against the two-channel board, and no improvement claim is made for either layer.

Separately, the coverage plan itself was re-examined at Will's request and amended where the re-examination found it wrong — mutation testing now precedes the handler registry (the original stub defense routed stubs into the one bucket the consumption ratchet deliberately never guards), mutation operators gained per-param perturbation and a derived-set rebuild hook, and the planned 58-dimension exploitability re-run is cancelled by measurement: a step-rule probe against a planted optimum showed one accepted step is worth 0.202 win-rate points against a 4.77-point resolution at the affordable budget, so the search moves to a 4–8-parameter reparameterization first. The full argument is `docs/COVERAGE-PLAN-REVIEW.md` . ABRA continues to have **no exploitability number**; `data/exploitability.json` remains void.

Taunt was not implemented, and the largest disagreement by pair volume was an engine fault rather than a harness one (3.50.0). The generated interaction matrix's disagreements were ranked by CARRIER uses x REACTOR uses — the pair's real frequency, not the carrier's — and the head of that list was `Taunt` , which appeared in twelve rows. The engine wrote the volatile, decremented it, and read it nowhere, so

a Taunted body still landed Hypnosis, Decorate, Strength Sap and another Taunt; Showdown's two handlers (`onDisableMove` at selection, `onBeforeMove` at execution) are now both wired off one derived table, `volatile` - > the move category it refuses . The row ranked #1 by volume, `partingshot` -> `throatchop` , had been filed as a probable staging artifact on the strength of a species mismatch; it was the engine — a pivot MOVE was given the bare-switch priority, so Parting Shot out-spiced everything and its replacement, not its user, took the incoming attack. Known disagreements fell from 94 to 72 (19 inside the scored set, 53 in buckets the gate discards). The census rose 211 -> 216 live of 219 probed; the damage differential did not move.

The mutation tier's defect count was wrong in the direction that inflates it, and the correction is recorded rather than quietly applied (3.49.1). The full sweep reported 97 DEFECT-CANDIDATE operators and an open total of 340; the two highest-usage rows were checked by hand and **both were false positives**. `damageMultAll` / `lifeorb` reads the tag for the damage and branches on the item's *name* only for the recoil — latent, not live. `halvesDamage` / `lightscreen` is not a defect at all: the engine ignores the tag's `mult` deliberately, because the artifact carries the singles value 0.5 and this is a doubles engine where the reduction is 2732/4096. A mutation verdict says what *moved*; it cannot see a deliberate override. The triage now grades every open operator **A/B/C/D from a parse of the frozen engine source** — A, no lookup for the tag and no branch on the carrier's name; B, the param is overridden by the engine's own constant; C, the behaviour is hardcoded by name; D, the param is read and this battery could not move it. **Nought of the 97 is class A.** The ratchet counts class A only (163 operators, 56 carrier × tag rows), because a number that counts false positives is a number people learn to ignore. Class A is *not* a count of missing mechanics — it says the fact reaches the simulator neither as a tag nor by name, and a third route (`mv.rc` , `data/move-effects.js` , an action kind) can still carry it — so the census's `armed` field is the second sort key and 49 of the 56 rows have no armed probe. The rule is gated on three cases decided by hand before it existed (Taunt A, Light Screen B, Life Orb C) and refuses to publish if it cannot reproduce them.

Outplayed turns are not noise: the click-censoring fix (3.42.0)

The policy fits learn from human clicks reconstructed out of replay logs. The log records what **happened**; the fit needs what was **clicked**. Across all **241,927 recorded human actions over 8,942 clean open-sheet games** (`data/policy-weights.json` , the fit corpus), **1,336 (0.5522%) were never clicks at all** and were being fitted as though they were. The classifier that decides which is which is measured separately, against the protocol's own annotations over 10,009 games (`data/click-censoring-census.json` , re-run 2026-08-05 on the current engine) — a slightly larger sweep than the fit, because the census reads every stored game while the fit takes only those it can build a board for:

class	n	share	mechanism
CLEAN	256,394	94.9530%	the recorded action is the click
PARTIAL	3,559	1.3180%	a redirector soaked the attack; the true target is one of two live foes
COERCED	1,475	0.5463%	Encore replaced the action (1,243); a phazing move dragged the mon in (232)
unreadable	7,668 + 888 + 38	3.1827%	unmatched, trivial, ambiguous

The shares are the reason this table can be re-run without re-arguing the fit: the census has now been taken three times as the store grew, and the three class shares agree to a hundredth of a point across all of them (`CHANGELOG.md` 3.42.0 and 3.47.0).

Both coerced classes were **invisible to every counter in the project**. The move Encore forces out is on the victim's own legal menu, so the matcher accepted it; and `|drag|` is stored with the same shape as `|switch|` , so a phazed arrival read as a voluntary switch decision. This is label noise, and learning with mislabelled examples is strictly harder than learning with missing ones (Natarajan, Dhillon, Ravikumar & Tewari, 2013). It is also **Missing Not At Random** in Rubin's (1976) sense — the corruption lands precisely on the turns where the opponent's play worked.

Coerced actions now leave the labelled set and are counted. Redirected attacks are kept under the **partial-label** likelihood (Cour, Sapp & Taskar, JMLR 2011): the contribution is the marginal $\log \sum_{c \in C} P_w(c \mid \text{board}, \text{choice set})$ over the candidate set, fitted by Generalized EM (Dempster, Laird & Rubin 1977; Neal & Hinton 1998), where the E-step is the responsibility $q_{c_i} = p_{c_i} / \sum_{c'} p_{c'}$ and the M-step is the existing conditional-logit gradient on q -weighted rows.

The estimator was validated on planted weights before the refit ran. Real corpus feature rows, synthetic labels drawn from a known w^* , the real censoring process applied to those labels, three seeds (data/partial-label-em.json):

regime	rows censored	$\ \hat{w} - w^* \ _2$ oracle	naive	EM	noise floor
heavy, systematic	20.96%	0.9978	1.8913	1.0208	0.2600
the corpus's own rate	0.44%	0.9978	0.9948	1.0021	0.2600

EM recovers **97.4%** of the censoring bias where the naive fit is visibly wrong, and at the rate the corpus actually censors the bias is **-0.0030 against a 0.2600 floor** — inside the noise.

Result, paired on 48,274 held-out decisions over 1,851 games, bootstrapped over GAMES (data/censoring-value.json , re-run 2026-08-05 on the current engine and the grown corpus; every figure below is inside the interval of the smaller 3.42.0 run it replaces — that run's numbers are in CHANGELOG.md 3.42.0 and docs/MEASURE.md §14, and the comparison is tabulated in §17):

held-out class	after – before	
COERCED (n=293): P(model picks the action no human chose)	-0.002613 [-0.003650, -0.001672]	clears zero
PARTIAL (n=650): mass on the true candidate set	+0.000122 [-0.000261, +0.000514]	contains zero
PARTIAL: log-likelihood of the candidate set	-0.002662 [-0.004002, -0.001368]	clears zero, worse
CONTROL, CLEAN (n=47,331): log-likelihood	+0.000485 [0.000189, 0.000777]	clears zero
CONTROL, CLEAN: top-1	-0.008 [-0.107, 0.085]	contains zero

Read plainly: **the fabricated labels are unlearned and the redirection correction bought nothing measurable.** Its own validation predicted that — the class is 1.35% of actions with a candidate set of exactly two, so there was almost no bias to remove. No corpus-wide top-1 improvement is claimed; the fix's justification is that a wrong label is a wrong label. Every effect is smaller than its class's split-half noise floor and resolves only because the comparison is paired per decision.

The mechanism is legible in the refit: of 58 weights, 9 moved past 2 SE and the largest single movement is `stallIntoEncore` — *"I am about to Protect and something across from me can Encore me for it"* — at **-1.0502 → -1.6281**. The poisoned rows were victims "choosing" their last move under an active Encore; deleting them makes clicking into an Encore threat look worse, which is the direction the mechanic predicts.

Three limits, stated. (i) The two vectors also differ by 86 games of corpus growth and by the refit itself, so the attribution rests on the weight-movement pattern rather than on an isolated control; `CENSORING=off` exists to run that control and has not been run. (ii) Priority-blocked attempts (Armor Tail, Queenly Majesty) are recoverable — the protocol names the attacker and the move in **299 of 299 cases (100.0%)** — but live only in raw logs covering 67.23% of the corpus, and the missing third is one archive, so recovering them would reweight the sample by source. (iii) `board.js` narrows the choice set for a Choice item and not for the `onDisableMove` family, so **a measured fraction of logged actions** were priced against a menu that had already shrunk — a wrong denominator rather than a wrong label, and a separate refit. *(The counts once printed here were from a census superseded on 2026-08-06 when engine/click_census.js was given an explicit corpus scope. They are NOT restated, because the artifact that would restate them —*

`data/censoring-value.json` — *refuses to regenerate: both weight vectors were fitted under the pre-WIRE-114 engine, so re-scoring them through the current one would measure the censoring change plus three wires at once. It clears with the refit. Read `data/click-censoring-census.json` for the current class counts.*)

A degenerate signature: when every arm of a controlled probe returns the same integer (3.56.0)

The accuracy subsystem was probed with the instrument this division already had — stage the mechanic, stage its absence, run both, difference the result. Six arms over four mechanics, ~5,000 combined uses in the store:

arm	with	without
Coil (+1 accuracy)	0	0
Minimize (+2 evasion on the target)	258	258
Wide Lens (×1.1)	0	0
Bright Powder (×0.9)	116	116
Sand Veil, in sand (×0.8)	115	115
No Guard (accuracy → true)	0	0

Exact equality across every arm is a stronger signal than a wrong number, and it is a different one. A miscalibrated modifier produces a difference of the wrong size; a difference of *identically zero*, repeated over four independently-implemented mechanics, is evidence about the **path**, not the **parameters**. Section "*A mechanic that fires everywhere*" (3.44.0) records the mirror-image signature — a rule firing on 100% of a population it should split — and both are instances of the same diagnostic: read the *distribution* of the controlled difference, not its mean.

Three unrelated defects were on that path, which is why no single hypothesis explained all six arms. (i) The Showdown→engine stat map sent `accuracy` and `evasion` to `null`; eleven boost applicers key off that map, so a payload of `{atk:+1, def:+1, accuracy:+1}` applied two of its three components and reported success. (ii) No item or ability was consulted for accuracy at any call site. (iii) The roll called `moveAccuracy(id, field)` — **a signature that admits no attacker and no defender**. Defects (i) and (ii) are omissions and are ordinary; (iii) is a *type-level* impossibility, and it is the one worth generalising from: a function whose parameters cannot express the question is unfalsifiable by any test that only checks its output. The census had graded the accuracy family LIVE on exactly that basis for as long as it had existed.

The repair is one authority, `hitChance(att, def, id, field, ctx)`, called at all four to-hit sites, with `printedAccuracy` preserving `true` (never-miss) as distinct from `100`, the standard $(3+n)/3$ Gen-III+ stage table [Bulbapedia, *Accuracy*], and the roll relocated **below** target resolution so a defender exists to interrogate. Direction is not hand-typed: `ACCURACY-MODIFIER CONFORMANCE` re-derives all 12 handlers from the live format object and takes sign from the hook name — 12 handlers, 13 rows, **0 disagreements**.

Separately, `Substitute` deducted 25% of the user's HP through the generic `costsUserHP` path and created no substitute: `playerAction` resolves the move to `kind: 'affect'`, so the `kind==='sub'` branch added in WIRE 42 was unreachable at the time it was written. 1,976 clicks in the store of an action **strictly dominated by passing** — a rare case where the correct baseline is not "a slightly worse policy" but "a negative-value action no rational agent takes", which makes any policy fitted over those turns miscalibrated in a *direction*, not merely in magnitude.

The bypass rule is likewise derived rather than reasoned. The intuitive encoding — *sound moves pass through a substitute* — is true and insufficient: the three highest-usage bypassing moves in this format are **Encore (4,848)**, **Taunt (1,503)** and **Disable (730)**, and none carries the `sound` flag. `SUBSTITUTE-BYPASS`

CONFORMANCE re-derives `bypasssub` across all 500 moves: **51 carried, 0 missing, 0 invented**. The general principle is the one this project states as *flags feed tags; match on tag shape, never on a name* — a semantic proxy for a mechanical flag will be right on the examples that motivated it and wrong on the tail that matters.

Reported rather than closed. Five tags are *absent*, not unprobed, and are declared with usage counts instead of being probed red — gate (c) ratchets on "every probe MISSING", so a red probe would have broken a ratchet to record a fact the (b) column already records. The largest, `ability|auraBoost` (5,663 uses), is a **representational** limitation rather than a missing branch: the multiplier is field-wide over the full roster and `dmgRange` is given two bodies and a field. Wiring it changes a `board.js` -facing input and is therefore a design decision, routed as one.

A mechanic that fires everywhere is not a mechanic that works (3.44.0)

Psychic Terrain refuses priority moves. The simulator knew that, and had known it since the tag artifact was first read. What it did not know is that the refusal only applies to a **grounded** target — so `priorityRefusedAbove` applied the terrain bar *outside* its own defender loop, without inspecting a single body:

```
for (const d of (defenders || [])) { /* the ability bar */ }
if (field && terrainId(field.terrain) === 'psychic') out = Math.min(out, 0);
```

The cost is concentrated on the most-clicked move in the format. **Fake Out (12,872 corpus uses)**, along with Extreme Speed, Sucker Punch, Aqua Jet, Ice Shard and Upper Hand, failed against every Flying type, every Levitate body and every Air Balloon whenever a Psychic Terrain was up.

The expected behaviour was taken from the official engine rather than from anyone's memory. Playing Incineroar's Fake Out into a Psychic Terrain raised by the opposing Indeedee's Psychic Surge, at the pinned commit under `gen9championsvgc2026regmb` :

target	official engine	damage
Garchomp, grounded	-activate move: Psychic Terrain	0
Orthworm, Earth Eater	-activate move: Psychic Terrain	0
Talonflame, Fire/Flying	-hint "doesn't affect airborne Pokémon"	237 → 216
Hydreigon, Levitate	-hint "doesn't affect airborne Pokémon"	251 → 233
Talonflame + Iron Ball	-activate move: Psychic Terrain	0

Two findings sit underneath the fix and matter more than it does.

The predicate existed three times, and none of the three was the one that mattered. Grounded-ness was written by hand in the entry-hazard block, in the switch-trapping branch, and in the Grassy Terrain heal — three copies that disagreed with each other about Iron Ball and about Eelevate. This is the failure mode CLAUDE.md names *facts are global*: the Grassy Terrain copy applied only the type half of the rule and **counted its own known-wrong half** in a failure counter. Somebody knew it was wrong, declared it, and the declaration outlived the reason for it — the derivation it said was unavailable had landed a release earlier. One `isGrounded(mon)` now answers the question for all four sites.

The census could not see the defect, because a scope is not a knob. The existing probe for this mechanic stages the block against a Garchomp, which is Dragon/Ground. It passes on the broken engine and on the fixed one. Every instrument in this division asks whether a mechanic *fires*; none asks whether it fires *only where it should*, and this is the fourth defect of that shape in two days. The replacement probe carries five arms, and the reason for each is that a smaller probe would have passed on some specific wrong engine —

including an **Earth Eater** arm, which is the reason the airborne ability set is a name rather than a shape read. The tempting artifact shape, `typeImmunity {type: 'Ground'}`, has three members: Levitate, Eelevate **and Earth Eater**. Orthworm is immune to Ground and firmly on the floor, and the official engine says so. Mechanics census **210 live of 213 probed**, 3 missing with written reasons, 0 hollow.

The matrix's own arithmetic is closed, and the coverage figure moves (3.43.0)

The interaction matrix is this project's largest conformance instrument, and until this release nothing checked its arithmetic. It printed a theoretical cross product, an emitted count and a ledger of named drops four lines apart, and **no code compared them**. They did not agree.

The identity is `theoretical === staged + dropped`, per axis, and it is now asserted at generation time rather than printed for a reader. It found three defects on its first run:

1. The denominator omitted the generator's own supplementary keys.

`tests/interaction_matrix.js` stages against `tags.linkage` MERGED with keys it derives itself; the theoretical total counted only the artifact's. 170 pairs were staged or dropped against a universe that had never heard of them. Theoretical **8,506 → 8,676**.

2. The type axis mis-costed its depth-cap tail by one.

The index was incremented before the cap was tested, so the tail excluded the very carrier the break was rejecting — 32 firings, 32 pairs of silence, in the direction that *flatters* the coverage rate.

3. The outcome buckets were not a partition.

`saturated` did not exclude a case that had thrown and `ko_timing` excluded nothing, so four cases were counted twice and the five printed totals summed to more than the number of cases run.

With the ledger closed, the generator recovers pairs it had been dropping unnamed: emitted **1,514 → 1,675**, live **899 → 1,031**.

The headline agreement figure falls, and that is the instrument working. The matrix reports **1,027 of 1,031 (99.6%)**, where the previous release reported 899 of 899 (100.0%). MEDICHAM did not regress: the four disagreements — Shield Dust against Fake Out, Throat Chop and Psychic Noise, and Steadfast against Upper Hand — sit on pairs the smaller generator never emitted. A 100.0% computed over a denominator that silently dropped 5,090 pairs was the less honest number. The four are `UNWIRED` rather than miscalculated: MEDICHAM's own two arms are identical on each, meaning the knob is absent rather than wrong.

The self-test is the point. `--selftest-reconcile` mis-costs exactly one drop by one pair — the smallest lie the ledger can tell — and requires the identity to stop the run. The file previously carried a header stating that the assertion fires; the assertion was defined and never called.

Layer 0 executes; the joint layer refits; the channel value is measured (3.41.0)

Same night, three division runs later. **Engine:** Layer 0 of the coverage plan is done — census **202 live of 205 probed, 3 missing with reasons**; interaction matrix **100.0%** (899 cases after retiring four redundant tags shrank the generated set from 1,012 — the retired facts live on under their surviving tags); the DEAD-tag ratchet fell **61 → 38**; the 26 orphan ability/item tags are triaged with a full disposition table in `ENGINE.md`. Two real bugs surfaced in passing and were fixed with probes shown failing first: the Intimidate retaliation arithmetic (Defiant read net +2 where the game gives +1; Competitive skipped the Attack drop) and Sheer

Force missing its $\times 1.3$ while its secondary-suppression half worked — strictly worse than no ability. The mutation tier's injection point (`__setDB` plus the derived-set rebuild hook the amended plan requires) landed and was exercised 26 times by the probe-red-demonstration harness.

Measure: the JOINT layer is refitted on the four-channel sheet — 95,886 usable joint turns, channel-reach counters at 99.7%, feature semantics verified — closing the second half of the fitting-environment gap. The held-out channel-value measurement ran A/B/C against the frozen two-channel incumbent (release `d3d04b669e18`), 44,982 paired decisions, 10,000 game-bootstrap resamples:

paired difference	logL/decision	top-1 points
information alone, weights frozen	+0.002853 [0.001611, 0.004072]	+0.009 [-0.140, +0.157]
refit, given the information	+0.002234 [0.001638, 0.002831]	+0.165 [0.029, 0.299]
everything vs what shipped	+0.005087 [0.003854, 0.006331]	+0.173 [-0.011, +0.360]

Split-half noise floor of the shipping arm: **0.331 top-1 points** (median, 20 cuts). The honest reading: the sheet channels buy a real per-decision likelihood gain — every logL interval clears zero — and **no demonstrable top-1 gain**; the one clearing interval is half the noise floor and resolves only because the comparison is paired. The first measurement attempt self-voided when the engine moved mid-run and was re-run clean — the release discipline working as designed.

And the tags regeneration was gated the way the rules demand: after the staged derivations landed in `data/tags.json`, `feature_fixture --check` confirmed both fitted weight vectors still agree with `board.js` on every fixture board — the new tags moved zero of the 58 feature columns, so the night's fits stand unre-run.

Measuring an engine that is being edited (3.36.0 – 3.39.0)

A refit that bought nothing, reported as such

The feature function was wrong about the weather on 10.72% of turn-boards: `engine/board.js` carried a private weather map that recognised Desolate Land and Primordial Sea — neither of which this format can produce — and did not recognise **sandstorm or snowscape**. Routing both reads through the engine's own exported `weatherId` moves **14 of 58 feature columns**, and touches a small single-digit percentage of vectors and decisions — consistent with the sand/snow share of the corpus. *(Exact counts withdrawn 2026-08-06: they were computed against a corpus superseded the same day, and the vectors they describe are among the three whose MEANING changed under the mega work — `switchSurvives1`, `switchKOSlow`, `switchDiesFirst`. They come back with the refit, measured, not restated from here.)*

Paired per decision on the same 46,162 held-out decisions across 1,772 games, bootstrapped over 10,000 game resamples:

paired difference	logL / decision	top-1 points
fix alone, weights frozen	+0.000348 [0.000075, 0.000623]	+0.048 [0.009, 0.093]
the refit, given fixed features	-0.000076 [-0.000172, +0.000021]	-0.074 [-0.155, +0.004]
everything vs what shipped	+0.000273 [-0.000010, +0.000556]	-0.026 [-0.117, +0.064]

Split-half noise floor for the refitted arm, 20 cuts: **median 0.192 top-1 points**. The fix is detectable *only* because the comparison is paired, and it is a quarter of that floor. **Refitting bought nothing** — the interval contains zero on both metrics, 1 of 58 weights moved beyond 2 SE, and the L2 of the whole weight change is 0.216. The fix was worth making because the feature function was wrong about the game, not because a metric improved; it did not need one and it did not get one.

The fitting environment is not the playing environment, and the gap is 20× the defect above

`engine/fit_policy.js:376` hands the board `{nature, item}`. `engine/magnemite.js:522` — the live player — hands it `{nature, item, ability, moves}`. Over 14,400 sheet entries in 1,200 games, **100.0% declare an ability and 100.0% declare four moves**, and the fit discards both.

	weather defect	sheet-channel gap
vectors that move	1,768 (0.75%)	WITHHELD (see below)
decisions that move	892 (2.78%)	WITHHELD (see below)
feature columns	14 of 58	20 of 58
games touched	238 (19.83%)	1,197 of 1,200 (99.75%)

TWO FIGURES WITHHELD 2026-08-15, AND THE REASON MATTERS MORE THAN THE NUMBERS. The two cells above read **37,460 (15.95%)** and **16,177 (50.47%)**. No artifact in this repository backs either one — `37,460` occurs in no file at all, and `16,177` occurs only in unrelated artifacts it has nothing to do with. They are withheld rather than captioned, because `CLAUDE.md` is explicit that a caption is not a quarantine and printing a figure with a caveat is the bug. **They were not caught by the currency gate for an unknown length of time, and the reason is a COLLISION.** `data/tags.json` happened to carry `"uses": 16177` for an unrelated tag, so the check "does this number appear in an artifact" answered yes. Regenerating the tags on 2026-08-14 moved 233 usage counts with the store, the coincidence evaporated, and the figure surfaced as untraceable. **It never had a source; it had a coincidence.** A traceability check that matches on a bare number will do this again, and that is now a register row rather than a footnote. The surrounding claim is NOT retracted:

`engine/fit_policy.js:376` hands the board `{nature, item}` while `engine/magnemite.js:522` hands it `{nature, item, ability, moves}`, and the 100.0% / 100.0% sheet-declaration rates above are separately sourced. What is withheld is the SIZE of the resulting gap, which needs a run that leaves an artifact.

The choice set is identical game for game, so this is purely what the board *knows*. **Half of every decision the fit trains on is priced against a board the player does not see.** This is `CLAUDE.md`'s fitting-vs-playing rule broken a second time and in the **opposite direction** from 2026-07-28 — the bot now sees *more* than the fit — which is precisely why nothing was watching for it. Not landed: it is one line plus a full refit, and it first needs a decision about the games where the opponent declines open team sheets, since a model fitted on four channels degrades differently from one fitted on two.

Interactions, generated rather than sampled

8,795 theoretical carrier × reactor pairs; 2,300 staged; **1,634 that can genuinely co-occur**, where co-occurrence is decided by the reference engine's own two arms differing rather than by our judgement — so "correctly blocked" stays distinguishable from "silently absent". The engine agrees on **1,614 of 1,634 (98.8%)**. Every pair the generator refuses is counted under a named reason and printed on each run. The 156 ordered persistent-field pairs each become an 8-turn script, which is the only construction that can observe *Trick Room was already up when Tailwind landed*; that axis went from 30/156 to **156/156**.

Validity: a measurement reads a frozen release

Three division agents ran concurrently with their files separated, and a 7,100-game exploitability run was still destroyed: the defender's own weight vector was refitted between the two legs, and the simulator showed four distinct content digests inside eight minutes. Nothing failed and nothing crashed.

The correction is not scheduling — serialising the divisions forfeits the parallelism they exist for. A measurement now opens an **immutable snapshot** (`engine/engine_release.js`) of every file whose content can change a reported number, the weights included, and reads those bytes rather than the live tree. The

membership of that set is declared as `SOURCES` in `engine/engine_release.js` and is read from there, never counted in prose: this sentence said **twelve** until 2026-08-22 and the declaration had grown well past it, each addition made because a release that was a valid digest set turned out not to be a loadable — then not a runnable — engine. It is a copy and not a checksum: verifying digests afterwards establishes only that the run was wasted. `engine/provenance.js` correspondingly stopped deciding staleness by **mtime** — the method this project's own rules discredit by name — and now compares content digests, honours a self-declared `void: true`, and prints how many artifacts still rest on timestamps alone (**0 verified, 92 by mtime**), ratcheted downward. On its first run the content check caught a rollout artifact computed against a version of its own generator that had since changed.

Consequently ABRA publishes no exploitability figure. The prior 63.2% [56.6, 69.3] is retracted on its own merits — 17 features against the 58 shipped, an engine 25 wire-fixes old, computed before the quality filter existed — and the re-run is void. One figure from the void run survives, because both of its legs fall inside a single stable window: the mirror control at **49.7% [46.2, 53.2]**, n=782, which retires the concern that an earlier 47.5% indicated a seat or pairing asymmetry rather than noise at n=217. A separate finding stands independently of the invalid tree: the attack **dies in 58 dimensions**, accepting 1 of 24 hill-climb steps against 10 of 18 at 17 features, so the step rule needs correcting before the re-run is worth its cost.